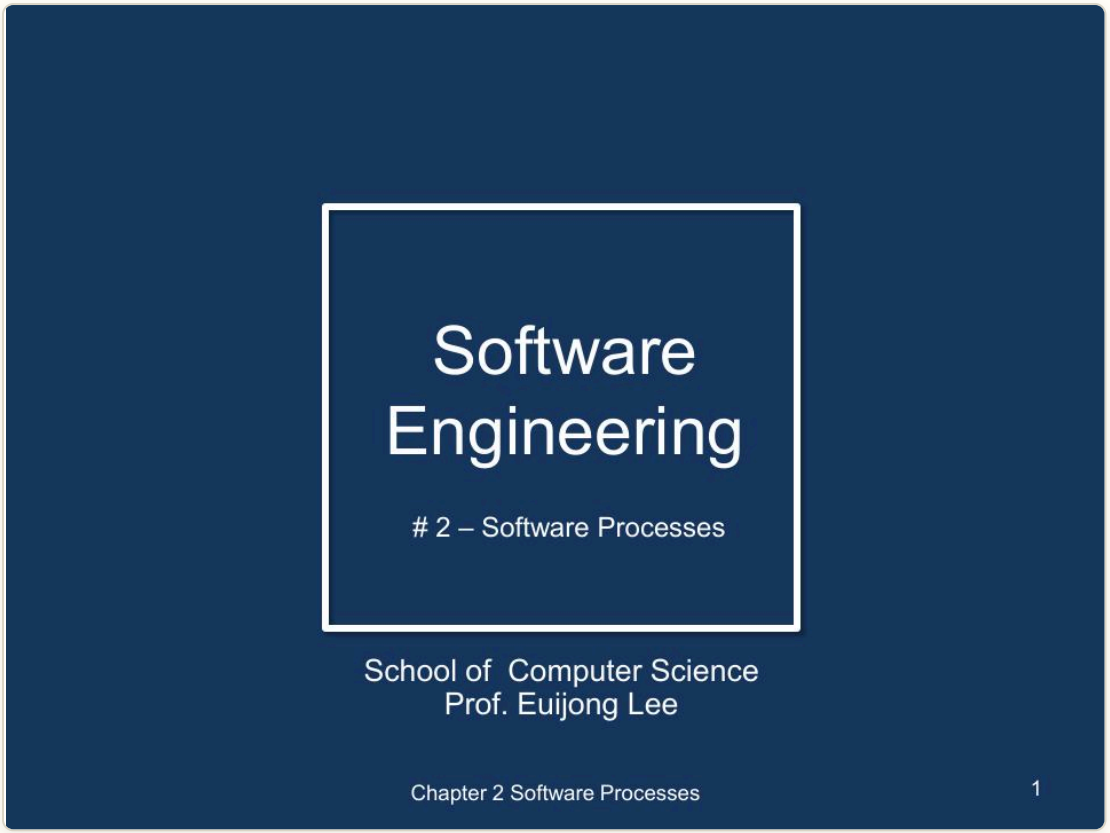
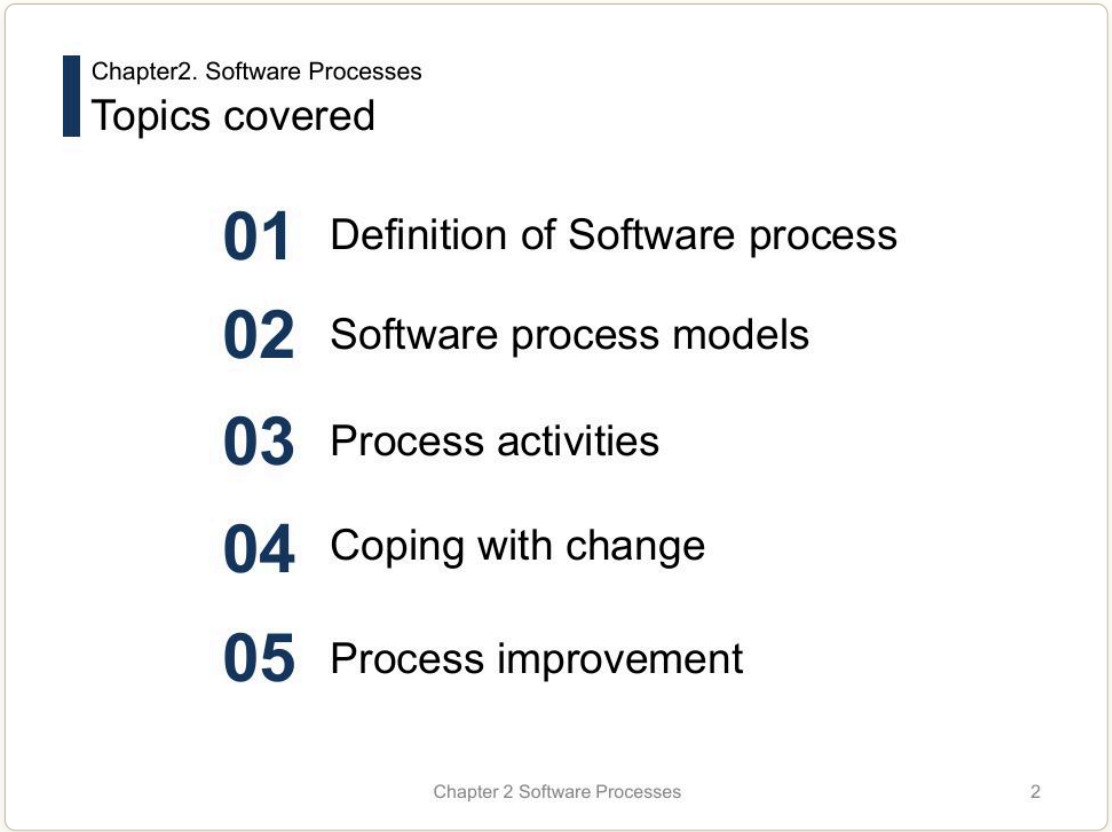


소프트웨어공학 #2 — 소프트웨어 프로세스

- 이의종 교수님

필기 ✍️

💡 이해 2장 흐름: 프로세스 정의 ⟶ 프로세스 모델 3가지 ⟶ 4대 활동 ⟶ 변경 대처 ⟶ 프로세스 개선(CMM)

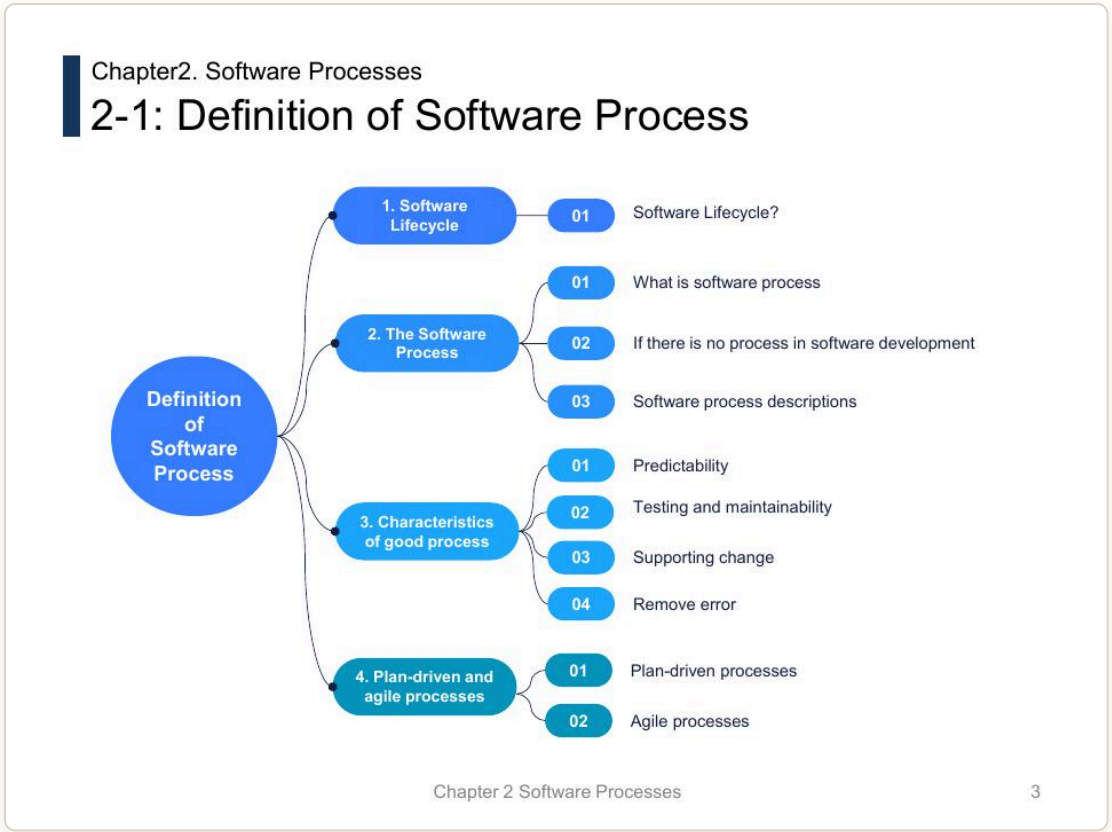


다루는 내용

- 01 SW 프로세스의 정의 / 02 SW 프로세스 모델 / 03 프로세스 활동 / 04 변경 대처 / 05 프로세스 개선

필기 ✍️

— 넘어가도 되는 쪽



2-1: SW 프로세스의 정의

- SW 생명주기 / SW 프로세스란 / 프로세스가 없으면
- 프로세스 기술
- 좋은 프로세스의 특징 — 예측 가능성, 테스트·유지보수 용이성, 변경 지원, 결함 제거
- 계획 주도 vs 애자일 프로세스

필기 ✍️

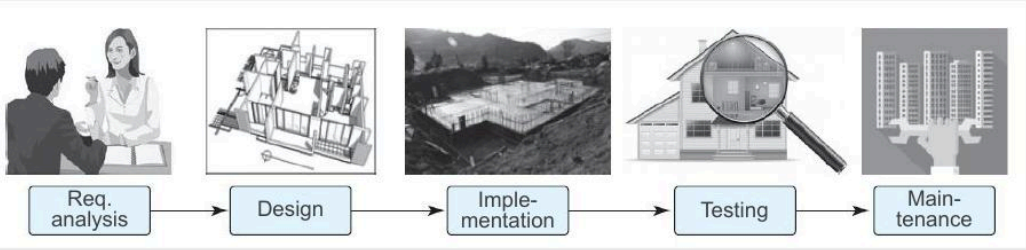
💡 이해 절 표지 — 아래 쪽들의 지도

Definition of software process

Software lifecycle

[Software lifecycle]

Technique and management-related works for software development



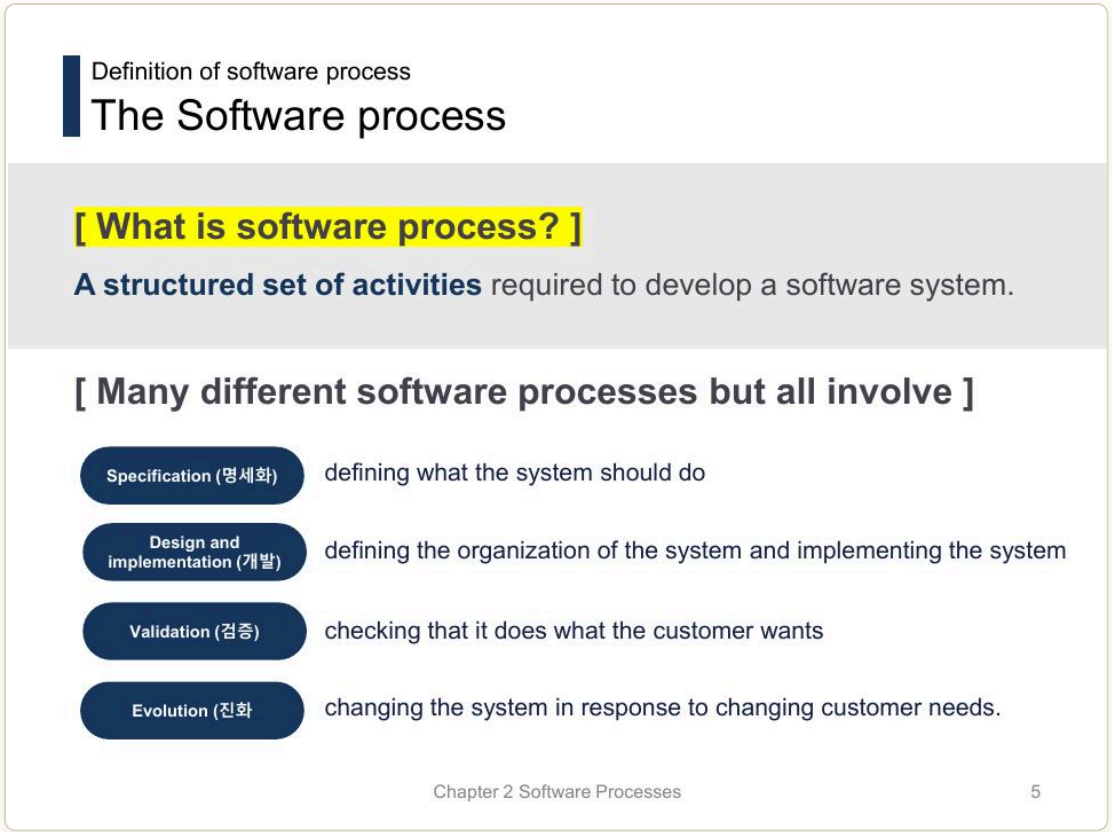
Source: <All about software engineering>. 생능출판사. 최은만

SW 생명주기 (Software lifecycle)

- SW 개발을 위한 **기술·관리 관련 작업**들
- 요구 분석 → 설계 → 구현 → 테스트 → 유지보수

필기 ✍️

— 넘어가도 되는 쪽



SW 프로세스란?

- **SW 시스템을 개발하는 데 필요한 구조화된 활동들의 집합**
- 프로세스는 여러 가지지만 모두 다음을 포함:
  - **Specification (명세화)** — 시스템이 무엇을 할지 정의
  - **Design and implementation (개발)** — 시스템 구조를 정하고 구현
  - **Validation (검증)** — 고객이 원하는 대로 하는지 확인
  - **Evolution (진화)** — 바뀌는 고객 요구에 맞게 시스템 변경

필기 ✍️

★ 시험 SW process = a structured set of activities required to develop a software system

★ 시험 4대 활동 (1장과 같다) — Specification · Design & implementation · validation · Evolution

Definition of software process

The Software process

- A software process model is an abstract representation of a process.
- It presents a description of a process from some particular perspective.

Chapter 2 Software Processes 6

SW 프로세스 모델

- **SW 프로세스 모델**은 프로세스의 **추상적 표현**
- 특정 **관점**에서 프로세스를 기술한다

필기 ✍️

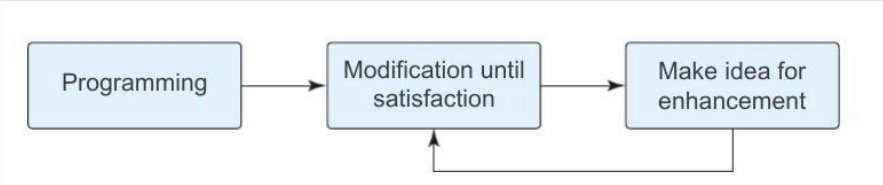
★ 시험 process model = an abstract representation of a process

Definition of software process

The Software process

[If there is no process in software development]

- Failing to realize the importance of designing.
- No plans, no work goals
- No awareness of the need for systematic testing work or quality assurance activities
- That called as code-and-fix (주먹구구식 방법)



Source: <All about software engineering>. 생능출판사. 최은만

Chapter 2 Software Processes 7

프로세스가 없으면?

- 설계의 중요성을 깨닫지 못함
- 계획도 작업 목표도 없음
- 체계적 테스트나 품질 보증 활동의 필요성을 모름
- 이것을 **code-and-fix(주먹구구식 방법)**라 한다
- 그림: 프로그래밍 → 만족할 때까지 수정 → 개선 아이디어 → 다시 프로그래밍 ...

필기 ✍️

★ 시험 프로세스 없음 = code-and-fix

Definition of software process

Software process descriptions

[When we describe and discuss processes, we usually talk about]

- the activities in these processes such as specifying a data model, designing a user interface, etc. and the ordering of these activities.

[Process descriptions may also include]

- **Products** (제품) : the outcomes(산출물) of a process activity.
- **Roles** (역할) : reflect the responsibilities of the people involved in the process.
- **Pre- and post-conditions** (사전, 사후조건)
  : statements that are true before and after a process activity has been enacted or a product produced.

Chapter 2 Software Processes 8

SW 프로세스 기술

- 프로세스를 말할 때는 보통 **활동**(데이터 모델 명세, UI 설계 등)과 그 **순서**를 이야기한다
- 프로세스 기술에는 다음도 들어갈 수 있다
  - **Products (제품)** — 프로세스 활동의 **산출물**
  - **Roles (역할)** — 참여자의 **책임**
  - **Pre- and post-conditions (사전·사후조건)** — 활동 전후나 산출물 생성 전후에 **참인 문장**

필기 ✍️

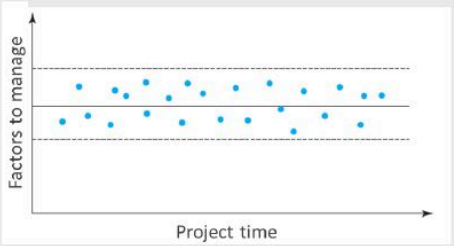
★ 시험 기술 요소 3개: Products · Roles · Pre/post-conditions

Definition of software process

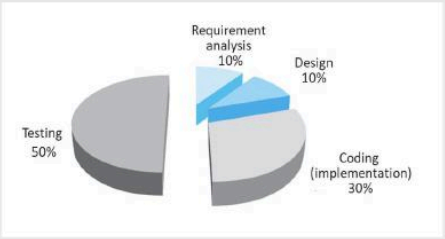
Characteristics of good process

[Characteristics of good process]

Predictability
(예측가능성)



Testing and maintainability
(테스팅과 유지보수의 용이성)



Source: <All about software engineering>. 생능출판사. 최은만

Chapter 2 Software Processes 9

좋은 프로세스의 특징 ①

- **Predictability (예측 가능성)**
- **Testing and maintainability (테스트·유지보수의 용이성)**

필기 ✍️

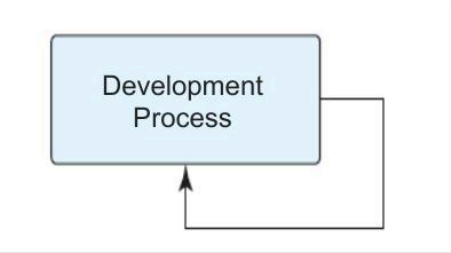
— 넘어가도 되는 쪽

Definition of software process

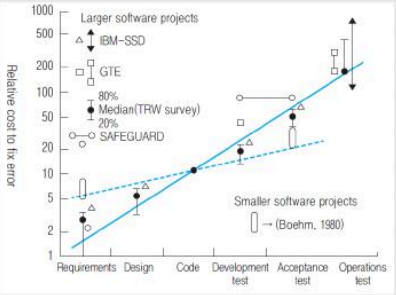
Characteristics of good process

[Characteristics of good process]

Supporting change (변경 지원)



Remove error (결함제거)



Source: <All about software engineering>. 생능출판사. 최은만

Chapter 2 Software Processes 10

좋은 프로세스의 특징 ②

- **Supporting change (변경 지원)**
- **Remove error (결함 제거)**

필기 ✍️

★ 시험 좋은 프로세스 4가지: 예측 가능성 · 테스트·유지보수 용이성 · 변경 지원 · 결함 제거

Definition of software process

Plan-driven and agile processes

[Plan-driven processes]

- processes where all of the process activities are planned in advance and progress is measured against this plan.

[Agile processes]

- planning is incremental and it is easier to change the process to reflect changing customer requirements.

✧ In practice, most practical processes include elements of both plan-driven and agile approaches.

✧ There are **no right or wrong software processes.**

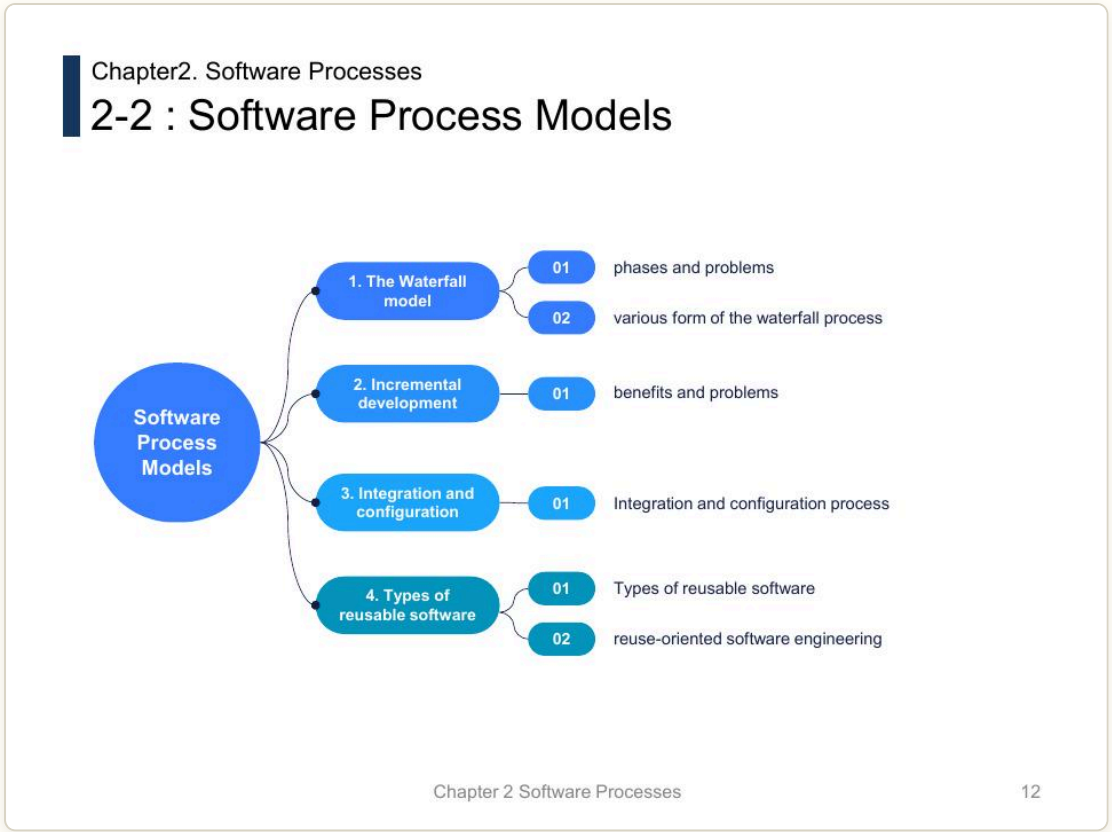
Chapter 2 Software Processes 11

계획 주도 vs 애자일 프로세스

- **Plan-driven processes**: 모든 활동을 **미리 계획**하고, 진척을 그 계획에 비추어 측정
- **Agile processes**: 계획이 **점진적(incremental)**이고, 바뀌는 고객 요구에 맞춰 프로세스를 바꾸기 쉽다
- 실제로 대부분의 프로세스는 **두 방식의 요소를 함께** 가진다
- **옳고 그른 SW 프로세스는 없다**

필기 ✍️

⚠ 주의 '애자일이 항상 낫다'는 틀림 — no right or wrong process

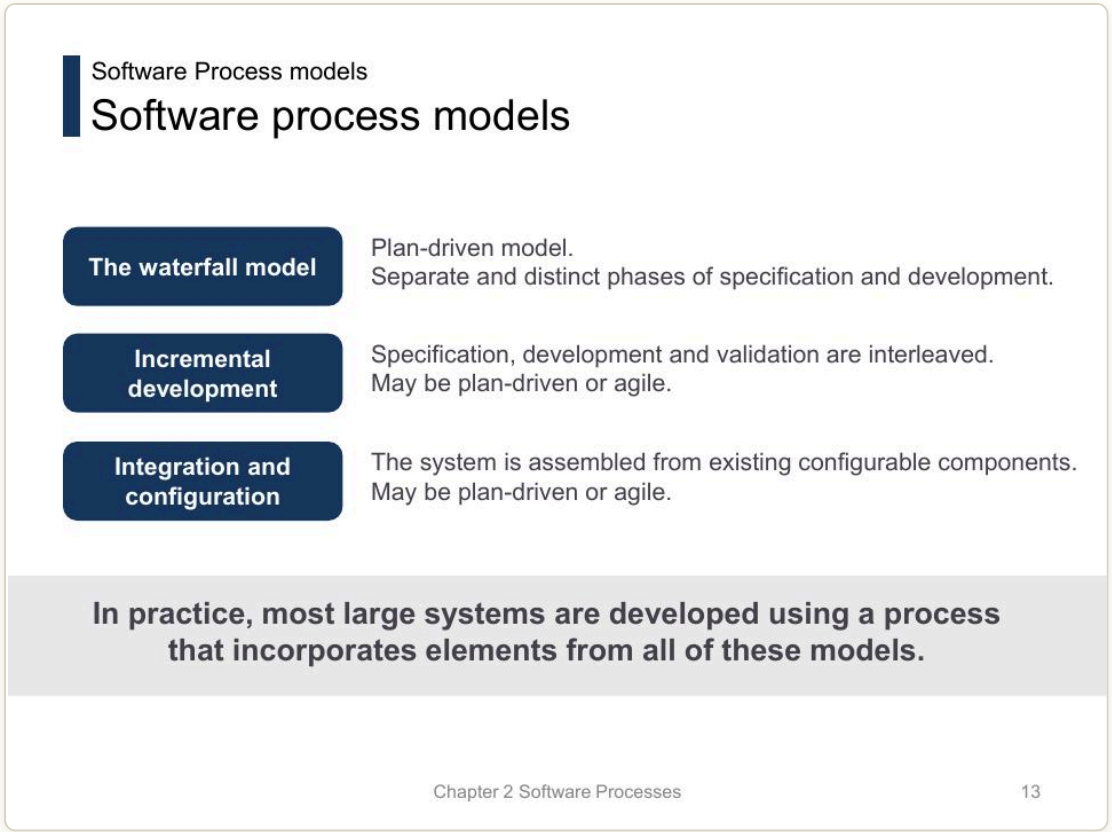


2-2: SW 프로세스 모델

- 폭포수 모델 — 단계와 문제, 여러 형태
- 점증적 개발 — 장점과 문제
- 통합과 구성 — 과정, 재사용 SW 종류, 재사용 지향 SE

필기 ✍️

💡 이해 절 표지 — 아래 쪽들의 지도



SW 프로세스 모델 3가지

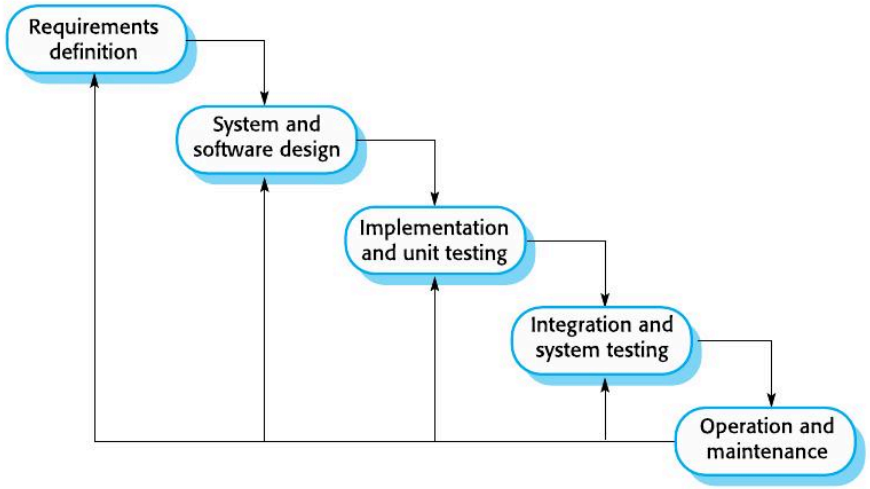
- **The waterfall model (폭포수)**
  - **계획 주도** 모델, 명세와 개발 단계가 **분리·구별**됨
- **Incremental development (점증적 개발)**
  - 명세·개발·검증이 **번갈아(interleaved)** 진행, 계획 주도일 수도 애자일일 수도
- **Integration and configuration (통합과 구성)**
  - 기존의 **구성 가능한 컴포넌트**로 시스템을 조립, 계획 주도일 수도 애자일일 수도
- 실제로 대형 시스템은 이 모델들의 요소를 **모두** 섞은 프로세스로 개발

필기 ✍️

★ 시험 모델 3개 + 각자 한 줄 특징 — 서술형 단골

Software Process models

The waterfall model



Chapter 2 Software Processes 14

폭포수 모델 (그림)

- Requirements definition → System and software design → Implementation and unit testing → Integration and system testing → Operation and maintenance
  - 위에서 아래로 한 단계씩 (폭포처럼)

필기 ✍️

— 넘어가도 되는 쪽

Software Process models

The waterfall model : phases

[There are separate identified phases in the waterfall model]

- Requirements analysis and definition
- System and software design
- Implementation and unit testing
- Integration and system testing
- Operation and maintenance

✧ The main **drawback** of the waterfall model is the **difficulty** of accommodating change after the process is underway

✧ In principle, a phase has to be complete before moving onto the next phase.

Chapter 2 Software Processes 15

폭포수 모델: 단계

- 분리된 단계들
  - 요구사항 분석·정의
  - 시스템·SW 설계
  - 구현과 단위 테스트
  - 통합과 시스템 테스트
  - 운영과 유지보수
- 주된 단점: 프로세스가 진행된 뒤에는 **변경을 받아들이기 어렵다**
- 원칙적으로 한 단계가 **끝나야** 다음 단계로

필기 ✍️

★ 시험 폭포수 5단계 순서 + 단점(변경 어려움)

Software Process models

The waterfall model : problems

[Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.]

- Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.
- Few business systems have stable requirements.

[The waterfall model is mostly used for large systems engineering projects where a system is developed at several sites.]

- In those circumstances, the plan-driven nature of the waterfall model helps coordinate the work.

Chapter 2 Software Processes 16

폭포수 모델: 문제

- 프로젝트를 **유연하지 않게** 단계로 나눠서 바뀌는 고객 요구에 대응하기 어렵다
  - 그래서 **요구사항이 잘 이해되고** 설계 중 변경이 적을 때만 알맞다
  - 안정된 요구사항을 가진 업무 시스템은 **드물다**
- 폭포수는 여러 장소에서 개발하는 **대형 시스템 공학 프로젝트**에 주로 쓰인다
  - 그런 상황에선 계획 주도라는 성격이 작업 조정에 도움

필기 ✍️

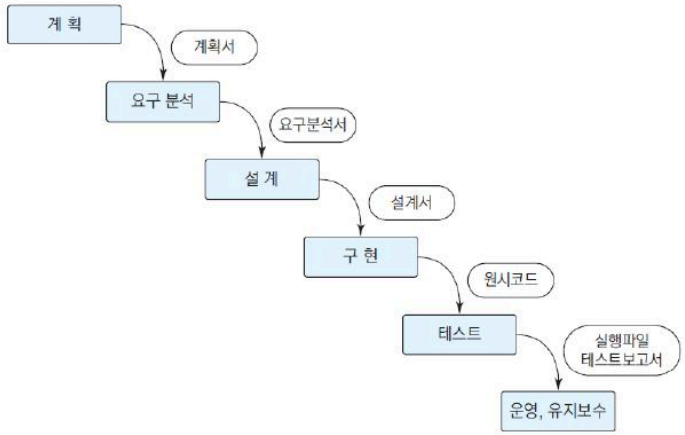
★ 시험 폭포수가 맞는 경우: 요구가 명확·안정 / 여러 곳에서 개발하는 대형 프로젝트

Software Process models

Various form of the waterfall process

[General form (as Korean)]

Source: <All about software engineering>. 생능출판사. 최은만



Chapter 2 Software Processes 17

폭포수의 여러 형태 — 일반형 (한국어)

- 계획 → 요구 분석 → 설계 → 구현 → 테스트 → 운영·유지보수
  - 단계마다 산출물: 계획서 · 요구분석서 · 설계서 · 원시코드 · 실행파일·테스트보고서

필기 ✍️

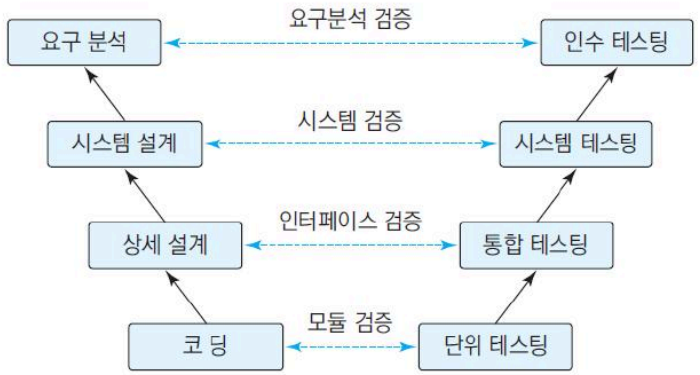
💡 이해 단계마다 문서가 나온다 = 계획 주도·문서 중심

Software Process models

Various form of the waterfall process

[V-model to enhance verification (as Korean)]

Source: <All about software engineering>. 생능출판사. 최은만



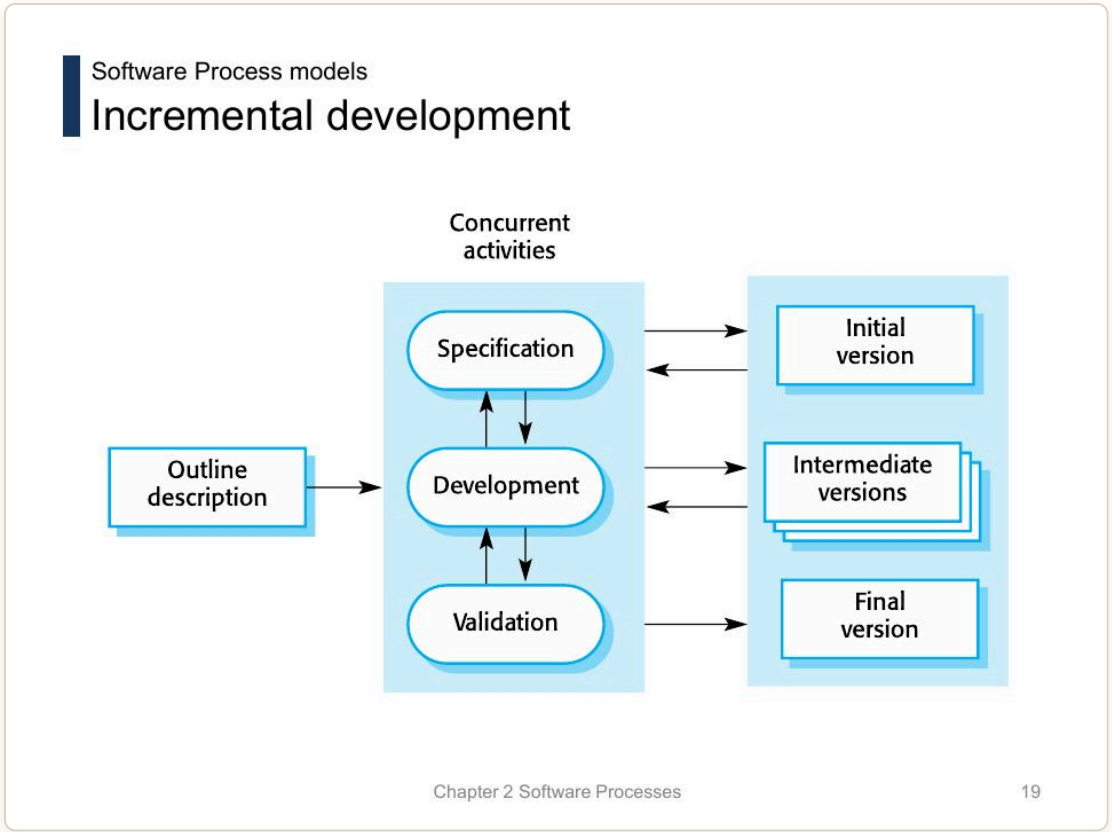
Chapter 2 Software Processes 18

폭포수의 여러 형태 — V 모델 (검증 강화)

- 왼쪽(내려감): 요구 분석 → 시스템 설계 → 상세 설계 → 코딩
- 오른쪽(올라감): 단위 테스팅 → 통합 테스팅 → 시스템 테스팅 → 인수 테스팅
  - 짝: 코딩↔단위(모듈 검증), 상세 설계↔통합(인터페이스 검증), 시스템 설계↔시스템 테스팅(시스템 검증), 요구 분석↔인수 테스팅(요구분석 검증)

필기 ✍️

★ 시험 V 모델 짝 4쌍 — 그림 그리기 대비



점증적 개발 (그림)

- 개요 설명 → 명세·개발·검증이 **동시에** 진행 → 초기 버전 → 중간 버전들 → 최종 버전

필기 ✍️

— 넘어가도 되는 쪽

Software Process models

Incremental development benefits

✧ **The cost of accommodating changing customer requirements is reduced.**

- The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.

✧ **It is easier to get customer feedback on the development work that has been done.**

- Customers can comment on demonstrations of the software and see how much has been implemented.

✧ **More rapid delivery and deployment of useful software to the customer is possible.**

- Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

Chapter 2 Software Processes 20

점증적 개발의 장점

- 바뀌는 고객 요구를 받아들이는 **비용이 줄어든다**
  - 다시 해야 할 분석·문서가 폭포수보다 훨씬 적다
- 지금까지 한 개발에 대한 **고객 피드백**을 받기 쉽다
  - 고객이 시연을 보고 얼마나 구현됐는지 확인
- 쓸모 있는 SW를 고객에게 **더 빨리 전달·배포**
  - 폭포수보다 일찍 SW를 쓰고 가치를 얻는다

필기 ✍️

★ 시험 장점 3개: 변경 비용↓ · 피드백 쉬움 · 빠른 전달

Software Process models

Incremental development problems

✧ **The process is not visible (가시적이지 못함).**

- Managers need regular deliverables to measure progress.
- If systems are developed quickly, it is not cost-effective to produce documents that reflect every version of the system.

✧ **System structure tends to degrade as new increments are added.**

- Unless time and money is spent on refactoring to improve the software, regular change tends to corrupt its structure.
- Incorporating further software changes becomes increasingly difficult and costly.

Chapter 2 Software Processes 21

점증적 개발의 문제

- 프로세스가 **보이지 않는다(not visible)**
  - 관리자는 진척을 재기 위해 정기 산출물이 필요한데
  - 빨리 개발하면 버전마다 문서를 만드는 게 비용 효율적이지 않다
- 증분이 더해질수록 **시스템 구조가 나빠진다**
  - 리팩토링에 시간·돈을 쓰지 않으면 잦은 변경이 구조를 망가뜨림
  - 추가 변경이 점점 어렵고 비싸진다

필기 ✍️

★ 시험 문제 2개: 가시성 부족 · 구조 저하

Software Process models

Integration and configuration

Integration and configuration process

- Based on **software reuse** where systems are integrated from existing components or application systems.
  (sometimes called COTS -Commercial-off-the-shelf systems)
- Reused elements may be configured to adapt their behaviour and functionality to a user's requirements.
- **Reuse is now the standard approach** for building many types of business system.

Chapter 2 Software Processes 22

통합과 구성 (Integration and configuration)

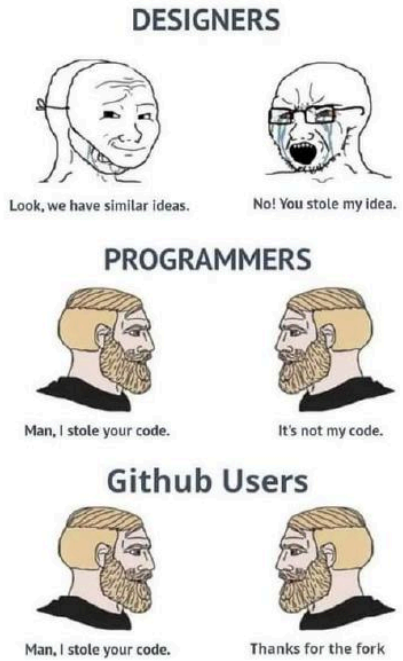
- **SW 재사용** 기반 — 기존 컴포넌트나 응용 시스템을 통합
  - (때로 **COTS** — Commercial-off-the-shelf systems)
- 재사용 요소를 사용자 요구에 맞게 동작·기능을 **구성(configure)**
- 재사용은 이제 많은 업무 시스템을 만드는 **표준 방식**

필기 ✍️

★ 시험 COTS = commercial-off-the-shelf (기성품 SW)

Software Process models

Integration and configuration



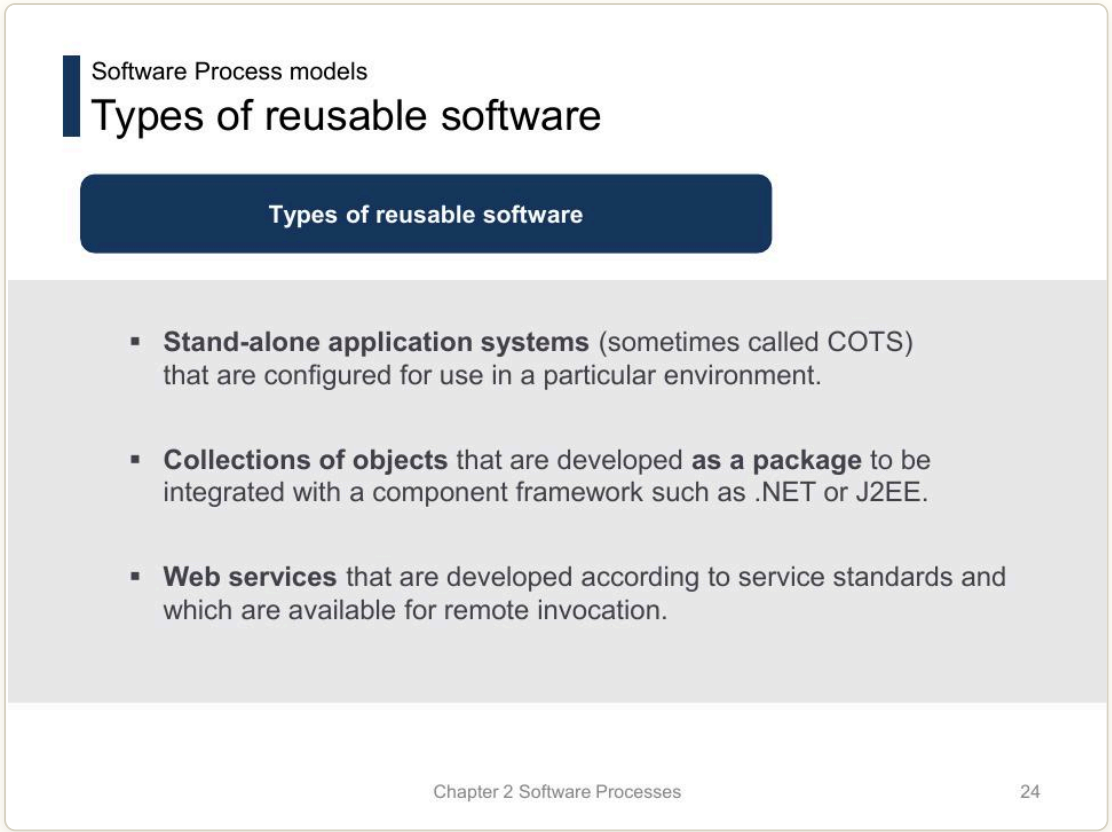
23

통합과 구성 (그림)

- 요구 명세 → SW 발견·평가 → 요구 정제 → 응용 시스템 구성 / 컴포넌트 적응·통합 → 시스템

필기 ✍️

— 넘어가도 되는 쪽

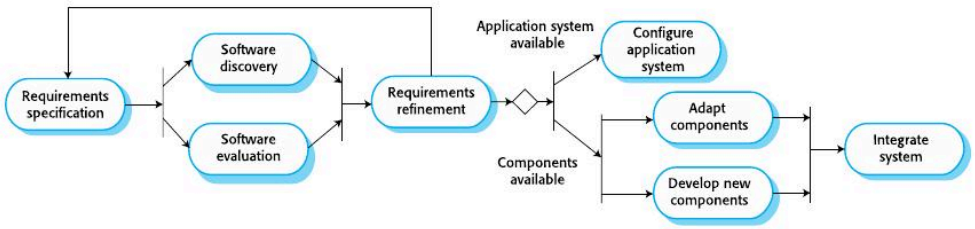


재사용 SW의 종류

- 특정 환경에 맞게 구성해 쓰는 **독립 응용 시스템** (COTS)
- .NET, J2EE 같은 **컴포넌트 프레임워크**에 통합되도록 패키지로 개발된 **객체 모음**
- 서비스 표준에 따라 개발돼 원격 호출 가능한 **웹 서비스**

필기 ✍️

★ 시험 재사용 SW 3종: COTS 응용 · 객체 모음(컴포넌트) · 웹 서비스

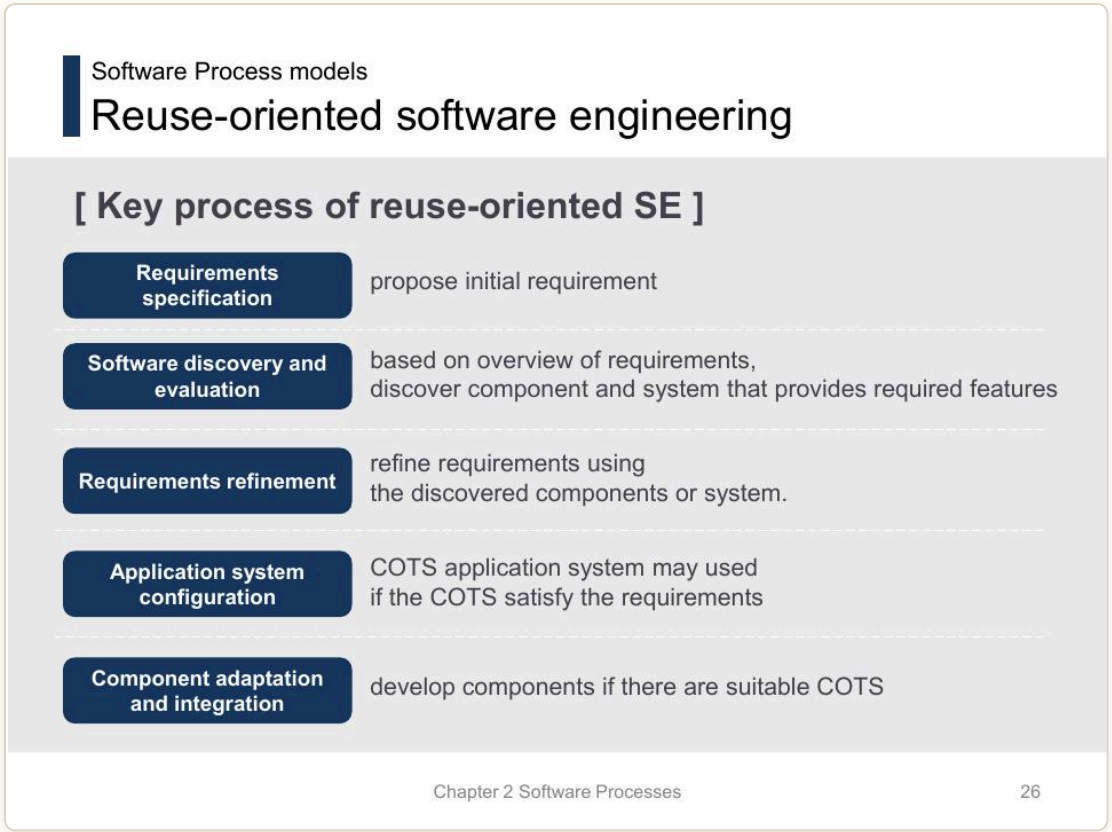


재사용 지향 SE (그림)

- 재사용 지향 SE의 과정 그림 — 다음 쪽에 단계 설명

필기 ✍️

— 넘어가도 되는 쪽



재사용 지향 SE의 핵심 과정

- **Requirements specification** — 초기 요구 제안
- **Software discovery and evaluation** — 요구 개요를 보고 필요한 기능을 주는 컴포넌트·시스템을 찾음
- **Requirements refinement** — 찾은 컴포넌트·시스템에 맞춰 요구를 다듬음
- **Application system configuration** — COTS가 요구를 만족하면 그대로 구성해 씀
- **Component adaptation and integration** — 알맞은 COTS가 없으면 컴포넌트를 개발·적응해 통합

필기 ✍️

💡 이해 보통의 개발과 달리 '찾고 → 요구를 맞춘다'는 순서가 특징

Software Process models

Reuse-oriented software engineering

[Advantages]

- **Reduced costs and risks** as less software is developed from scratch
- **Faster delivery** and **deployment** of system

[Disadvantages]

- Requirements compromises are inevitable so system may **not meet real needs of users**
- **Loss of control** over evolution of reused system elements

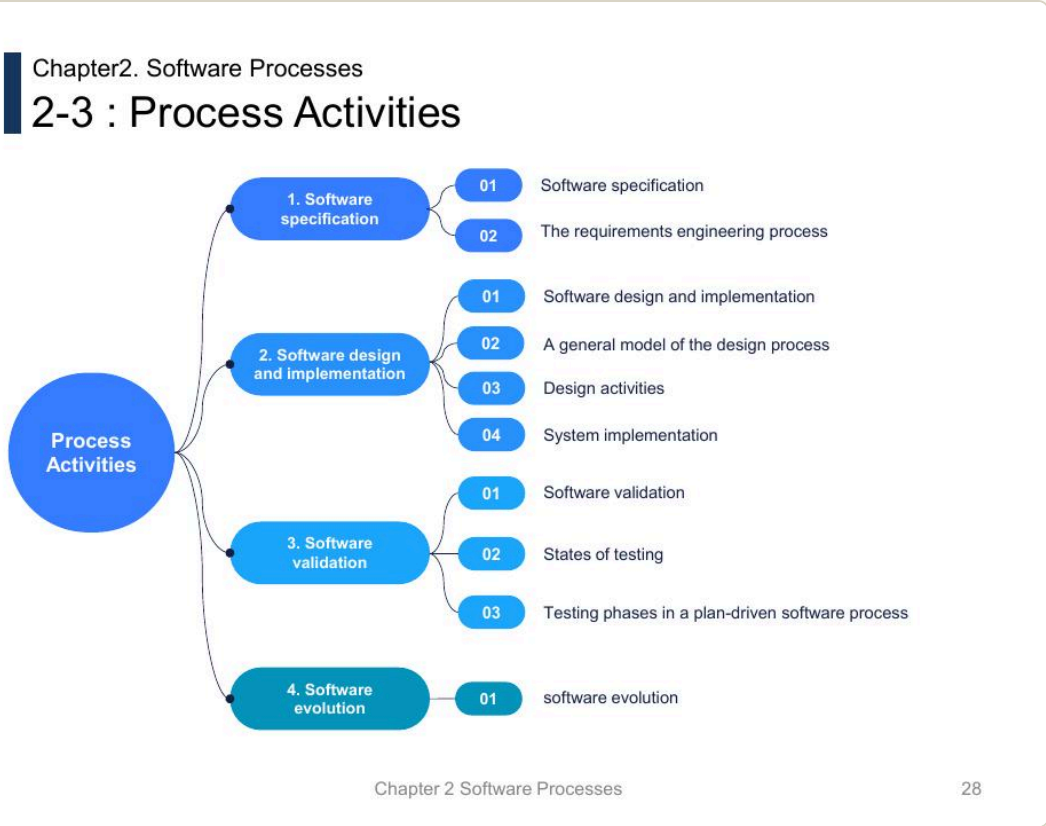
Chapter 2 Software Processes 27

재사용 지향 SE의 장단점

- **장점**
  - 처음부터 만드는 SW가 적어 **비용과 위험 감소**
  - **더 빠른** 전달과 배포
- **단점**
  - 요구사항 **타협이 불가피** → 사용자의 진짜 요구를 못 맞출 수 있음
  - 재사용 요소의 **진화를 통제할 수 없다**

필기 ✍️

★ 시험 단점 2개(요구 타협, 진화 통제 상실)가 시험에 잘 나옴



2-3: 프로세스 활동

- SW 명세 — 요구공학 과정
- SW 설계와 구현 — 설계 과정 모델, 설계 활동, 시스템 구현
- SW 검증 — 테스트 단계, 계획 주도 프로세스의 테스트(V 모델)
- SW 진화

필기 ✍️

💡 이해 절 표지 — 아래 쪽들의 지도

Process activities

Process activities

- Real software processes are **inter-leaved** sequences of technical, collaborative and managerial activities with the overall goal of specifying, designing, implementing and testing a software system.
- The **four basic process activities** of 1) specification, 2) development, 3) validation and 4) evolution are organized differently in different development processes.
- For example, in the waterfall model, they are organized in sequence, whereas in incremental development they are interleaved.

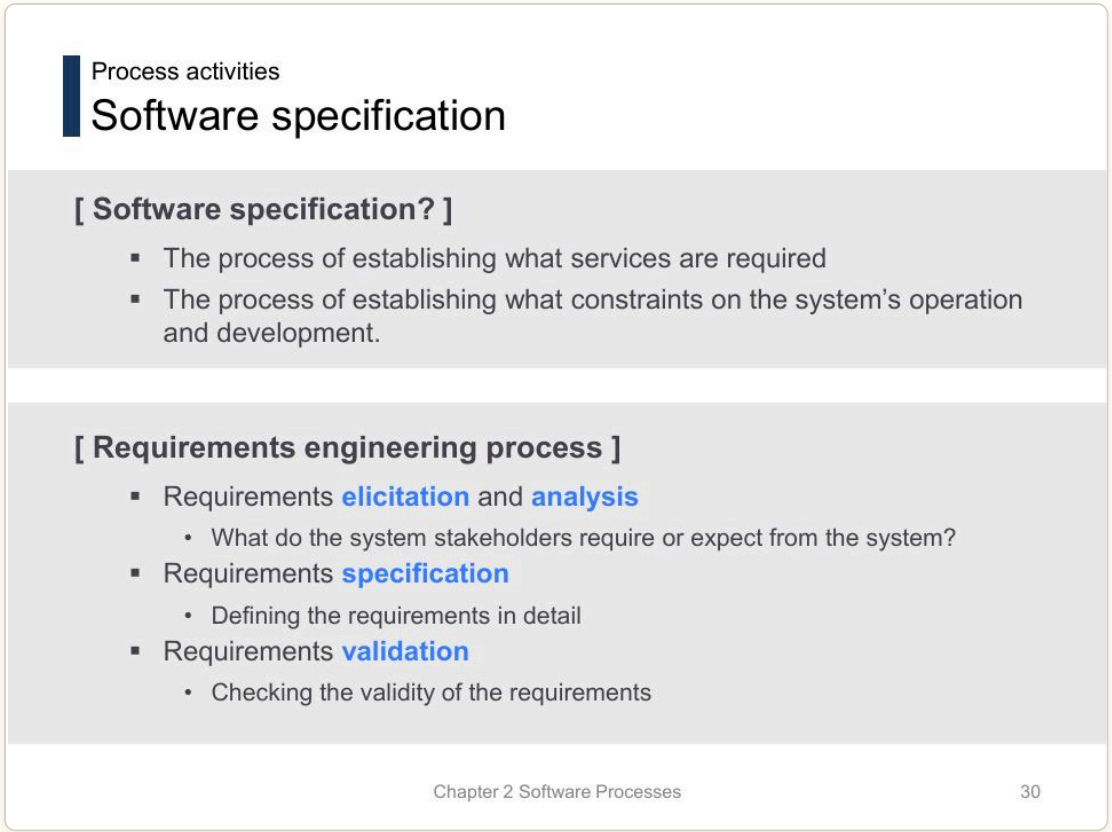
Chapter 2 Software Processes 29

프로세스 활동

- 실제 SW 프로세스는 기술·협업·관리 활동이 **얽힌** 순서 — 목표는 SW를 명세·설계·구현·테스트하는 것
- 4대 기본 활동 1) 명세 2) 개발 3) 검증 4) 진화는 프로세스마다 **다르게 조직**된다
  - 예: 폭포수는 **차례대로**, 점증적 개발은 **번갈아**

필기 ✍️

— 넘어가도 되는 쪽

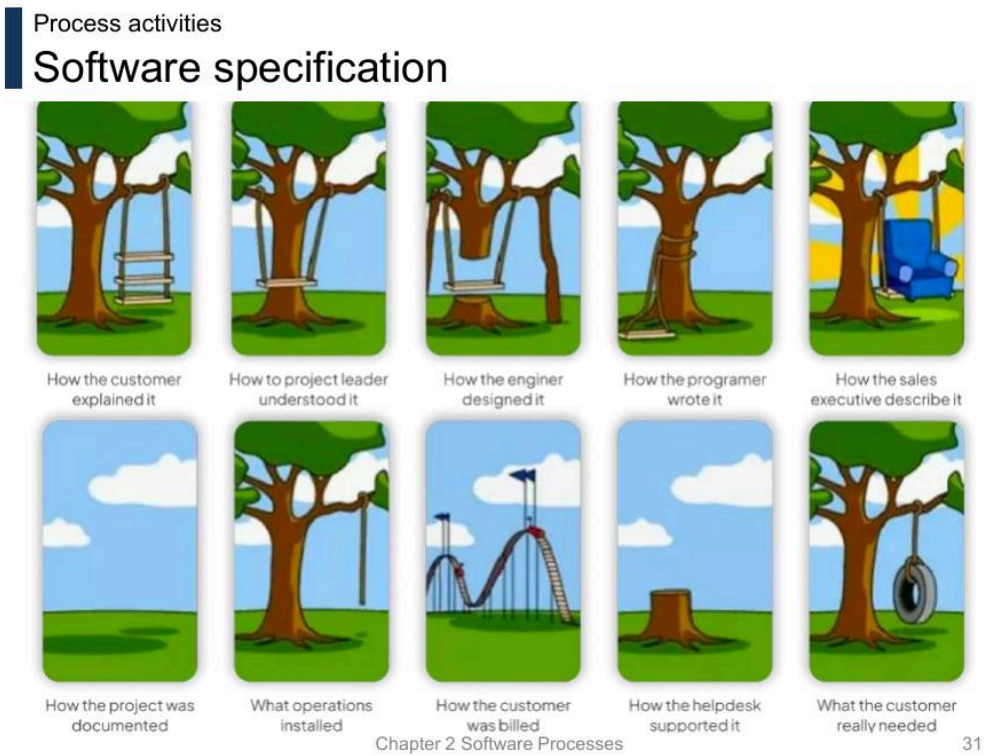


SW 명세 (Software specification)

- **어떤 서비스**가 필요하고, 시스템 운영·개발에 **어떤 제약**이 있는지 정하는 과정
- **요구공학 과정 (Requirements engineering process)**
  - **Requirements elicitation and analysis (도출과 분석)** — 이해관계자가 시스템에 무엇을 요구·기대하나?
  - **Requirements specification (명세)** — 요구사항을 자세히 정의
  - **Requirements validation (검증)** — 요구사항이 타당한지 확인

필기 ✍️

★ 시험 요구공학 3단계: 도출·분석 → 명세 → 검증



SW 명세 (그림)

- 요구공학 과정 그림 — 도출·분석 / 명세 / 검증이 이어지고, 결과로 시스템 기술·사용자·시스템 요구사항·요구사항 문서

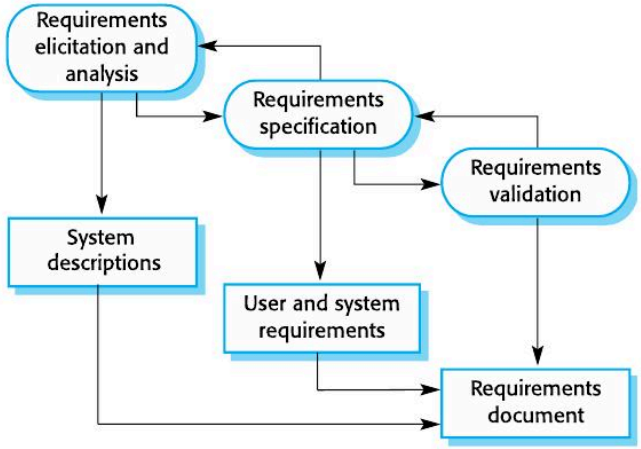
필기 ✍️

— 넘어가도 되는 쪽

Process activities

Software specification

[The requirements engineering process]



Chapter 2 Software Processes 32

요구공학 과정 (그림)

- 요구공학 과정의 단계와 산출물

필기 ✍️

⚠ 주의 6·7주차 요구 분석 상세 자료는 받지 못함 — 이 그림 수준만

Process activities

Software design and implementation

[Software design and implementation]

- The process of converting the **system specification into an executable system**.
- Software design
  - **Design a software** structure that realises the specification.
- Implementation
  - Translate this structure into **an executable program.**
- The activities of design and implementation are closely related and may be inter-leaved.

Chapter 2 Software Processes 33

SW 설계와 구현

- 시스템 명세를 **실행 가능한 시스템으로 바꾸는** 과정
  - **SW 설계** — 명세를 실현하는 SW 구조를 설계
  - **구현** — 그 구조를 실행 가능한 프로그램으로 옮김
  - 설계와 구현은 밀접하고 **번갈아** 진행될 수 있다

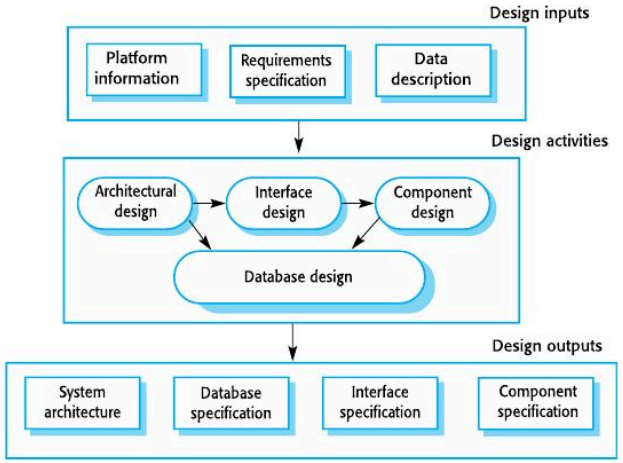
필기 ✍️

— 넘어가도 되는 쪽

Process activities

Software design and implementation

[A general model of the design process]



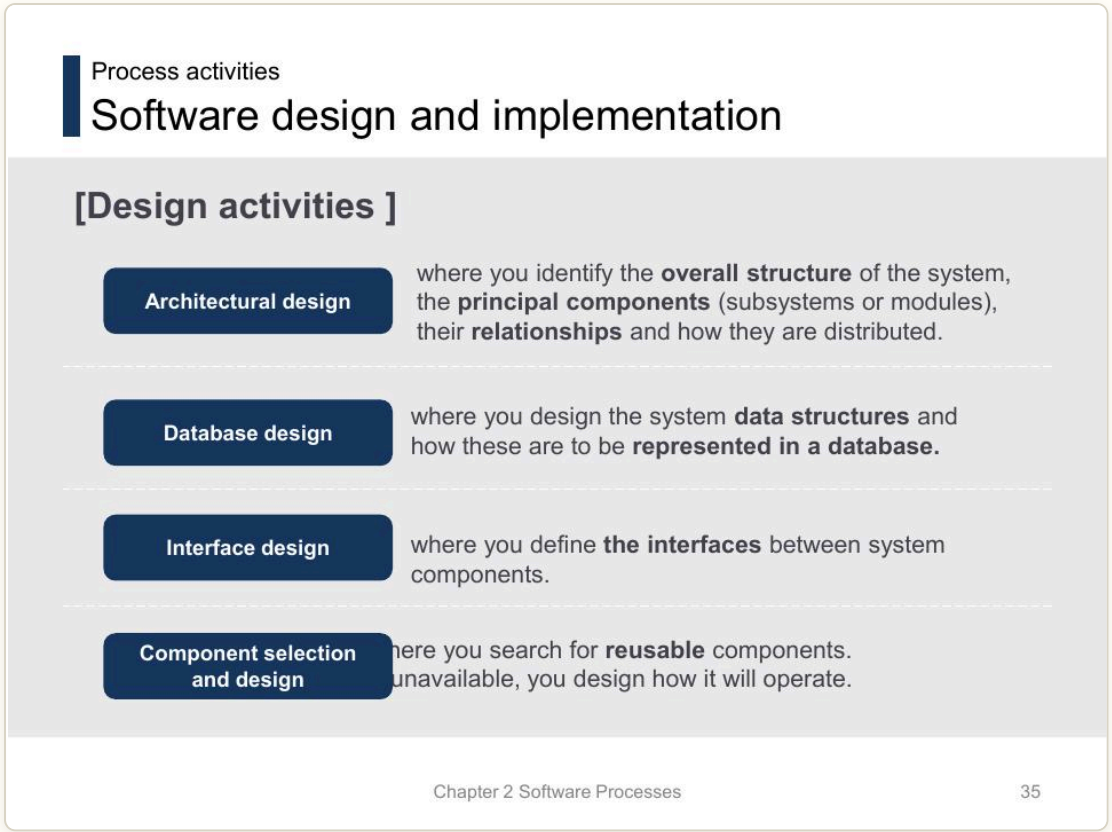
Chapter 2 Software Processes 34

설계 과정의 일반 모델 (그림)

- 입력(플랫폼 정보, 요구 명세, 데이터 기술) → 설계 활동(아키텍처·인터페이스·컴포넌트·DB 설계) → 산출물(시스템 아키텍처, DB 명세, 인터페이스 명세, 컴포넌트 명세)

필기 ✍️

— 넘어가도 되는 쪽



설계 활동

- **Architectural design** — 시스템 전체 구조, 주요 컴포넌트(하위 시스템·모듈), 그 관계와 배치
- **Database design** — 시스템 데이터 구조와 DB 표현
- **Interface design** — 시스템 컴포넌트 사이 인터페이스 정의
- **Component selection and design** — 재사용 컴포넌트를 찾고, 없으면 동작 방식을 설계

필기 ✍️

★ 시험 설계 활동 4개: 아키텍처 · DB · 인터페이스 · 컴포넌트 선택·설계

Process activities

Software design and implementation

[System implementation]

- The software is **implemented** either by developing a program or programs or by configuring an application system.
- Design and implementation are **interleaved activities** for most types of software system.
- **Programming** is an individual activity with **no standard process**.
- **Debugging** is the activity of finding program faults and correcting these faults.

시스템 구현

- SW는 프로그램을 개발하거나 응용 시스템을 구성해 구현
- 대부분 SW에서 설계와 구현은 번갈아 진행
- **프로그래밍은 개인 활동**이고 **표준 프로세스가 없다**
- **디버깅** = 프로그램 결함을 찾아 고치는 활동

필기 ✍️

⚠ 주의 '프로그래밍에는 표준 프로세스가 있다' → X

Process activities

Software validation

[Software validation]

- Verification and validation (V & V, 검증 및 확인)
  - intended to show that a **system conforms to its specification**
  - **meets the requirements** of the system customer.
- Involves **checking** and **review** processes and system **testing**.
- System testing involves **executing the system** with test cases that are derived from the specification of the real data to be processed by the system.
- **Testing** is the most commonly used **V & V activity.**

Chapter 2 Software Processes 37

SW 검증 (Validation)

- **V & V (Verification and validation, 검증 및 확인)**
  - 시스템이 **명세에 맞고(conform)**
  - **고객 요구를 만족함**을 보이려는 것
- 검사·검토 과정과 시스템 테스트를 포함
- 시스템 테스트 = 실제 처리할 데이터 명세에서 뽑은 **테스트 케이스**로 시스템을 실행
- **테스트가 가장 흔한 V&V 활동**

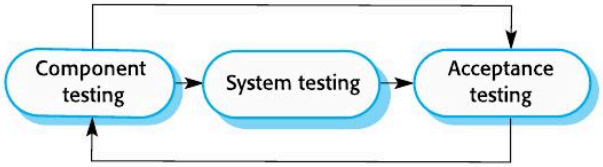
필기 ✍️

★ 시험 verification = 명세대로 만들었나 / validation = 고객이 원하는 걸 만들었나

Process activities

Software validation

[States of testing]



Component testing	: Individual components are tested independently. : Components may be functions or objects or coherent groupings of these entities.
System testing	: Testing of the system as a whole. : Testing of emergent properties is particularly important.
Customer testing	: Testing with customer data to check that the system meets the customer's needs.

Chapter 2 Software Processes 38

테스트 단계

- **Component testing** — 개별 컴포넌트를 **독립적으로** 테스트 (함수·객체·그 묶음)
- **System testing** — 시스템 **전체** 테스트, 창발적 성질(emergent properties) 테스트가 특히 중요
- **Customer testing** — **고객 데이터**로 고객 요구를 만족하는지 확인

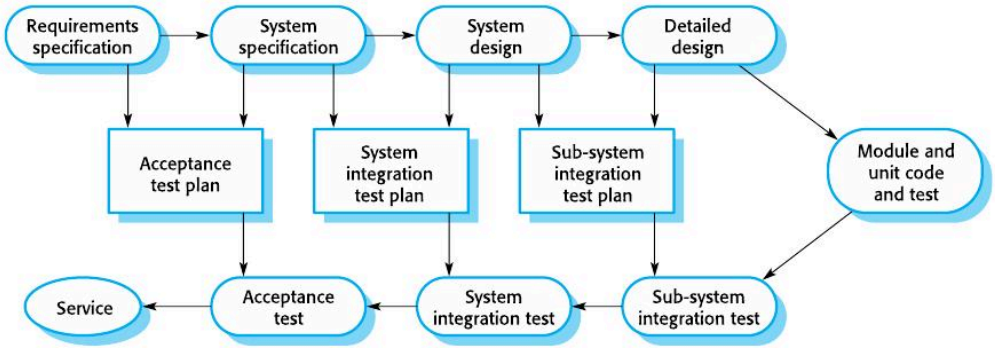
필기 ✍️

★ 시험 테스트 3단계: component → System → customer

Process activities

Software validation

[Testing phases in a plan-driven software process (V-model)]



Chapter 2 Software Processes 39

계획 주도 프로세스의 테스트 단계 (V 모델 그림)

- 요구 명세 → 인수 테스트 계획, 시스템 명세 → 통합 테스트 계획, 시스템 설계 → 하위 시스템 통합 테스트 계획, 상세 설계 → 모듈·단위 코드와 테스트
- 아래에서 위로: 하위 시스템 통합 테스트 → 시스템 통합 테스트 → 인수 테스트 → 서비스

필기 ✍️

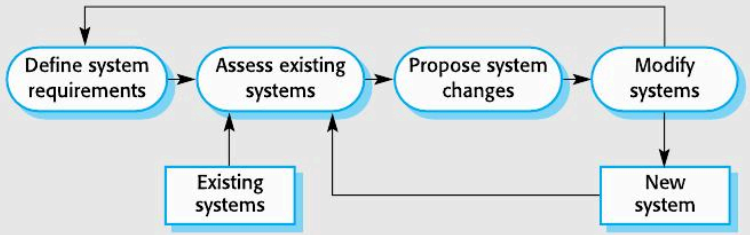
💡 이해 설계 단계에서 이미 테스트 계획을 세운다 = 계획 주도

Process activities

Software evolution

[Software evolution]

- Software is inherently **flexible and can change**.
- As **requirements change** through changing business circumstances, the software that supports the business must also evolve and change.
- Although there has been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new.



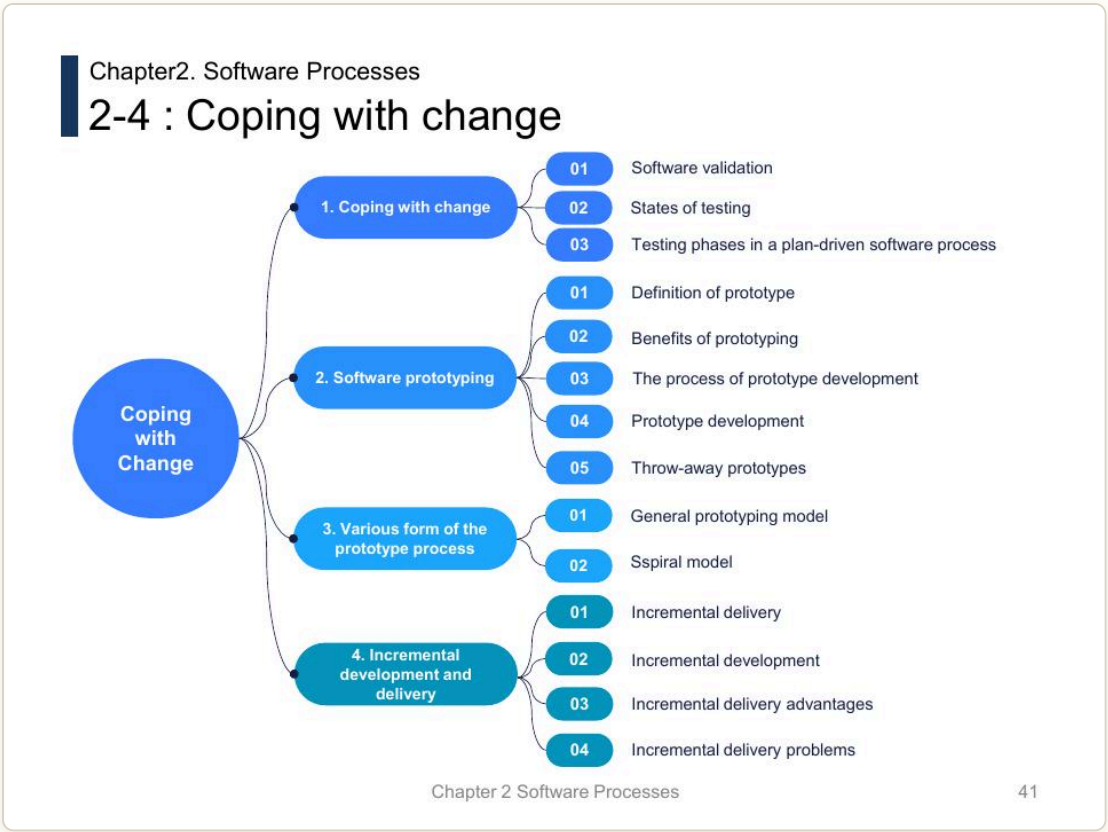
Chapter 2 Software Processes 40

SW 진화 (Software evolution)

- SW는 본래 **유연하고 바뀔 수 있다**
- 비즈니스 상황 변화로 요구가 바뀌면 그 업무를 돕는 SW도 진화·변화해야
- 개발과 진화(유지보수) 사이에 경계가 있었지만, **완전히 새로운 시스템이 점점 줄어서** 그 구분은 점점 무의미

필기 ✍️

★ 시험 개발 vs 유지보수 구분은 점점 무의미 (increasingly irrelevant)



2-4: 변경 대처 (Coping with change)

- 변경 대처
- SW 프로토타이핑 — 정의, 장점, 개발 과정, 버리는 프로토타입
- 프로토타입 프로세스의 여러 형태 — 일반 모델, 나선형 모델
- 점증적 개발과 인도 — 장점과 문제

필기 ✍️

💡 이해 절 표지 — 아래 쪽들의 지도

Coping with change

Coping with change : 변경 처리

✧ Change is inevitable in all large software projects.

- **Business changes** lead to new and **changed system** requirements
- **New technologies** open up new possibilities for improving implementations
- **Changing platforms** require application changes

✧ Change leads to **rework** so the **costs of change** include both rework (e.g., re-analysing requirements) as well as the costs of implementing new functionality

Chapter 2 Software Processes 42

변경 대처: 변경 처리

- 모든 대형 SW 프로젝트에서 **변경은 피할 수 없다**
  - 비즈니스 변화 → 새롭거나 바뀐 요구
  - 새 기술 → 구현을 개선할 새 가능성
  - 플랫폼 변화 → 응용 변경 필요
- 변경은 **재작업(rework)**을 부른다 — 변경 비용 = 재작업(요구 재분석 등) + 새 기능 구현 비용

필기 ✍️

— 넘어가도 되는 쪽

Coping with change

Coping with change

[Reducing the costs of rework]

Change anticipation (변경 예측)
the software process includes activities that can anticipate **possible changes before** significant rework is required.

- For example, a prototype system may be developed to show some key features of the system to customers.

Change tolerance (변경 허용)
the process is designed so that **changes can be accommodated** at **relatively low cost**.

- This normally involves some form of incremental development.
- Proposed changes may be implemented in increments that have not yet been developed.
- If this is impossible, then only a single increment (a small part of the system) may have be altered to incorporate the change.

Chapter 2 Software Processes 43

재작업 비용 줄이기

- **Change anticipation (변경 예측)**
  - 큰 재작업이 필요해지기 **전에** 가능한 변경을 예측하는 활동을 포함
  - 예: 핵심 기능을 고객에게 보여 주는 **프로토타입**
- **Change tolerance (변경 허용)**
  - 변경을 비교적 **적은 비용**으로 받아들이도록 프로세스를 설계
  - 보통 **점증적 개발** 형태
  - 제안된 변경을 아직 개발 안 한 증분에 구현
  - 불가능하면 증분 하나(작은 부분)만 고침

필기 ✍️

★ 시험 변경 예측 = 프로토타입 / 변경 허용 = 점증적 개발

Coping with change

Coping with change

[Coping with changing requirements]

- **System prototyping**
  - A version of the system or part of the system is **developed quickly** to **check the customer's requirements** and the feasibility of design decisions.
  - This approach supports change anticipation.

- **Incremental delivery**
  - System increments are delivered to the customer for **comment and experimentation.**
  - This supports both change avoidance and change tolerance.

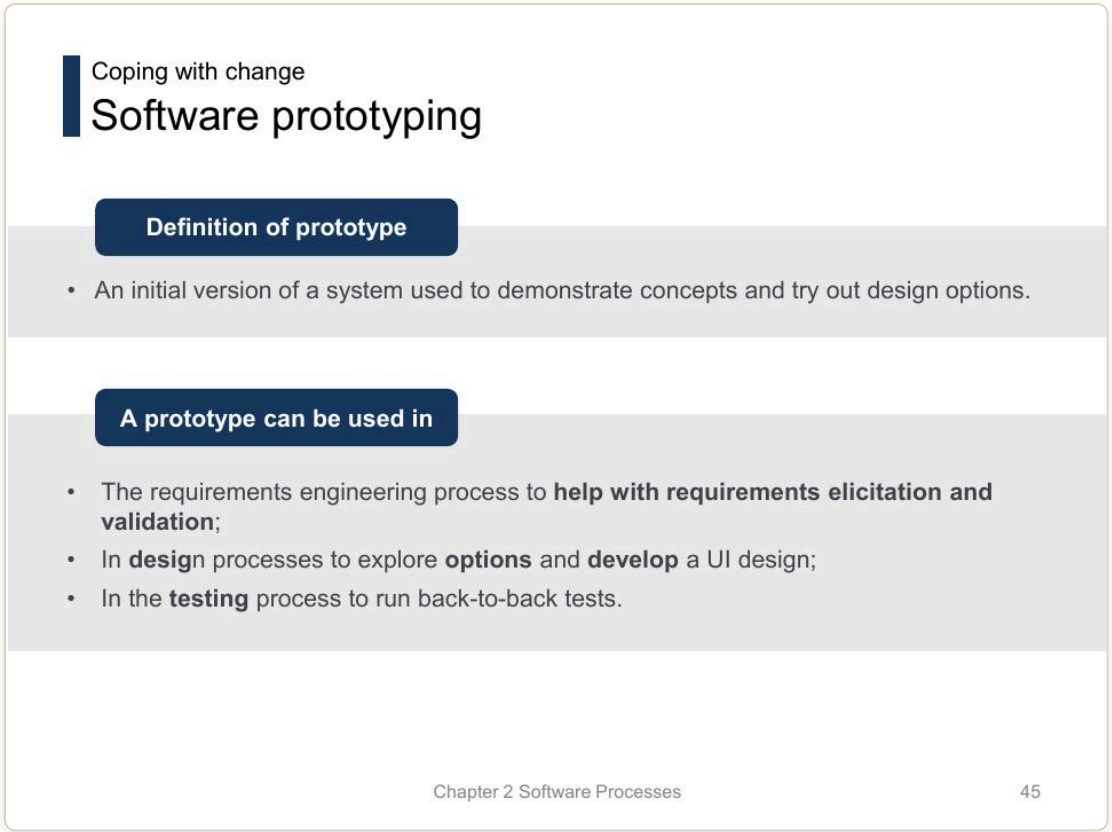
Chapter 2 Software Processes 44

바뀌는 요구에 대처하기

- **System prototyping** — 시스템(또는 일부)을 빨리 만들어 고객 요구와 설계 결정의 타당성 확인
  - **변경 예측**을 지원
- **Incremental delivery** — 시스템 증분을 고객에게 전달해 의견과 실험을 받음
  - **변경 회피와 변경 허용**을 둘 다 지원

필기 ✍️

— 넘어가도 되는 쪽



SW 프로토타이핑 — 정의

- **Prototype**: 개념을 보여 주고 설계 옵션을 시험해 보는 **시스템의 초기 버전**
- 쓰이는 곳
  - **요구공학** 과정 — 요구 도출과 검증
  - **설계** 과정 — 옵션 탐색, UI 설계
  - **테스트** 과정 — back-to-back 테스트

필기 ✍️

★ 시험 프로토타입 쓰임 3곳: 요구공학 · 설계 · 테스트

Coping with change

Software prototyping

[Benefits of prototyping]

- Improved system **usability**.
- A closer match to users' **real needs**.
- Improved **design quality**.
- Improved **maintainability**.
- **Reduced** development **effort**.

Chapter 2 Software Processes 46

프로토타이핑의 장점

- 시스템 **사용성** 향상
- 사용자의 **진짜 요구**에 더 가깝게
- **설계 품질** 향상
- **유지보수성** 향상
- **개발 노력** 감소

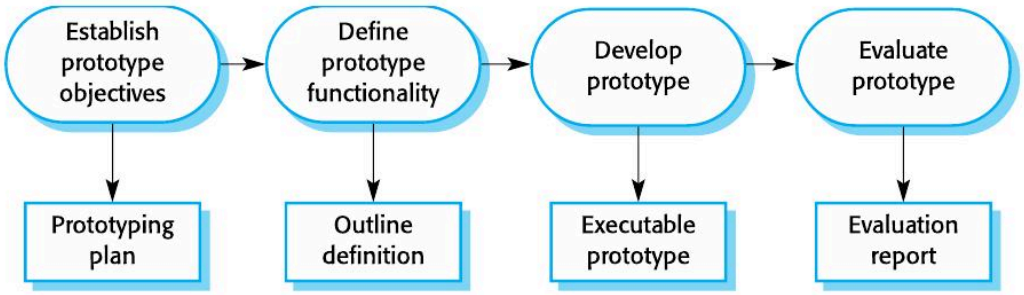
필기 ✍️

★ 시험 장점 5개 (사용성·요구 적합·설계 품질·유지보수성·노력 감소)

Coping with change

Software prototyping

[The process of prototype development]



Chapter 2 Software Processes 47

프로토타입 개발 과정 (그림)

- 목표 설정 → 기능 정의 → 프로토타입 개발 → 프로토타입 평가
  - 산출물: 프로토타이핑 계획 → 개요 정의 → 실행 가능한 프로토타입 → 평가 보고서

필기 ✍️

— 넘어가도 되는 쪽

Coping with change

Software prototyping

[Prototype development]

- May be based on **rapid prototyping languages or tools**
- May involve **leaving out functionality**
  - Prototype should focus on areas of the product that are not well-understood;
  - Error checking and recovery may not be included in the prototype;
  - Focus on functional rather than non-functional requirements such as reliability and security

Chapter 2 Software Processes 48

프로토타입 개발

- 빠른 프로토타이핑 언어나 도구를 쓸 수 있다
- 기능을 **빼고** 만들 수 있다
  - 잘 이해되지 않은 부분에 집중
  - 오류 검사·복구는 넣지 않을 수 있다
  - 신뢰성·보안 같은 비기능 요구보다 **기능 요구**에 집중

필기 ✍️

— 넘어가도 되는 쪽

Coping with change

Software prototyping

[Throw-away prototypes]

- Prototypes should be **discarded** after development as they are not a good basis for a production system:
  - It may be impossible to tune the system to meet non-functional requirements;
  - Prototypes are normally undocumented;
  - The prototype structure is usually degraded through rapid change;
  - The prototype probably will not meet normal organizational quality standards.

Chapter 2 Software Processes 49

버리는 프로토타입 (Throw-away prototypes)

- 프로토타입은 개발 뒤 **버려야** 한다 — 실제 시스템의 좋은 기반이 아니므로
  - 비기능 요구에 맞게 조정하기 불가능할 수 있음
  - 보통 **문서가 없다**
  - 빠른 변경으로 구조가 보통 **나빠져** 있다
  - 조직의 일반 품질 표준을 못 맞출 것

필기 ✍️

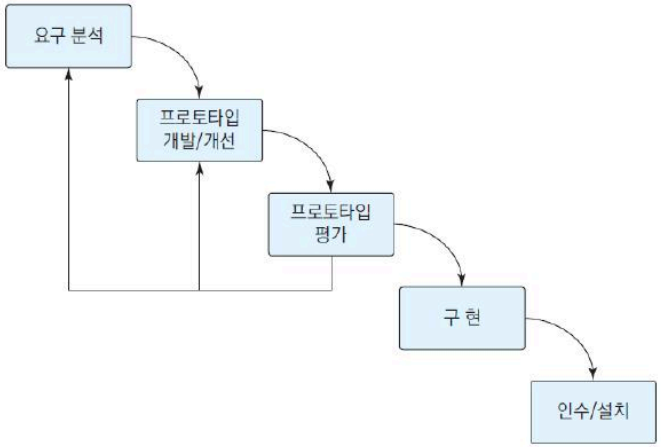
★ 시험 버리는 이유 4개 — 비기능 조정 불가 · 문서 없음 · 구조 저하 · 품질 표준 미달

Coping with change

Various form of the prototype process

[General prototyping model (as Korean)]

Source: <All about software engineering>. 생능출판사. 최은만



Chapter 2 Software Processes 50

프로토타입 프로세스 — 일반 모델 (한국어)

- 요구 분석 → 프로토타입 개발·개선 → 프로토타입 평가 → (만족할 때까지 되돌아감) → 구현 → 인수·설치

필기 ✍️

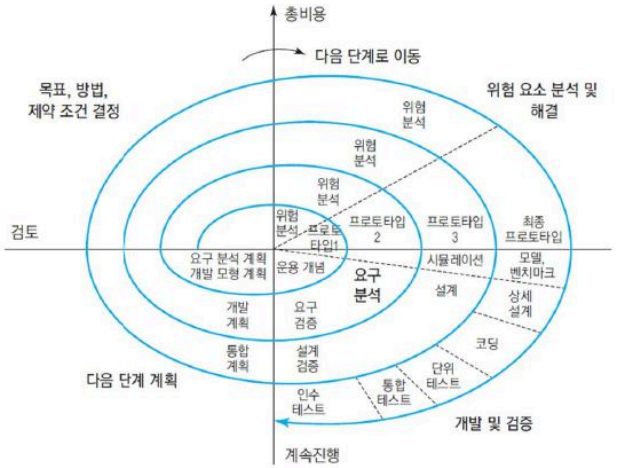
— 넘어가도 되는 쪽

Coping with change

Various form of the prototype process

[Spiral model (as Korean)]

Source: <All about software engineering>. 생능출판사. 최은만



Chapter 2 Software Processes 51

프로토타입 프로세스 — 나선형 모델 (한국어)

- 한 바퀴 = 4사분면: **목표·방법·제약 결정 → 위험 요소 분석·해결(프로토타입) → 개발·검증 → 다음 단계 계획**
- 바퀴를 돌수록 총비용이 늘고, 프로토타입 1 → 2 → 3 → 최종 프로토타입
- 검토 후 다음 단계로 이동

필기 ✍️

★ 시험 나선형 모델의 핵심 = 매 바퀴마다 '위험 분석'

Coping with change

Incremental delivery

[Incremental delivery (단계적 인수)]

- Rather than deliver the system as a single delivery, the **development** and **delivery** is **broken down** into increments with each increment delivering part of the required functionality.
- User **requirements** are **prioritised**, and the highest priority requirements are included in early increments.
- Once the development of an increment is started, the **requirements** are **frozen** though requirements for later increments can continue to evolve.

Chapter 2 Software Processes 52

점증적 인도 (Incremental delivery)

- 시스템을 한 번에 전달하지 않고, 개발·전달을 **증분**으로 나눔 — 증분마다 기능 일부를 전달
- 사용자 요구에 **우선순위**를 매기고, 가장 높은 것을 **앞 증분**에
- 한 증분 개발을 시작하면 그 요구는 **동결(frozen)** — 뒤 증분의 요구는 계속 진화 가능

필기 ✍️

★ 시험 우선순위 높은 것 먼저 + 진행 중 증분의 요구는 동결

Coping with change

Incremental development and delivery

[Incremental development]

- **Develop** the system in **increments** and **evaluate** each **increment** before proceeding to the development of the next increment;
- Normal approach used in **agile** methods;
- Evaluation done by user/customer proxy.

[Incremental delivery]

- Deploy an increment for use by end-users;
- More **realistic evaluation** about practical use of software;
- **Difficult** to implement for replacement systems as increments have less functionality than the system being replaced.

Chapter 2 Software Processes 53

점증적 개발 vs 점증적 인도

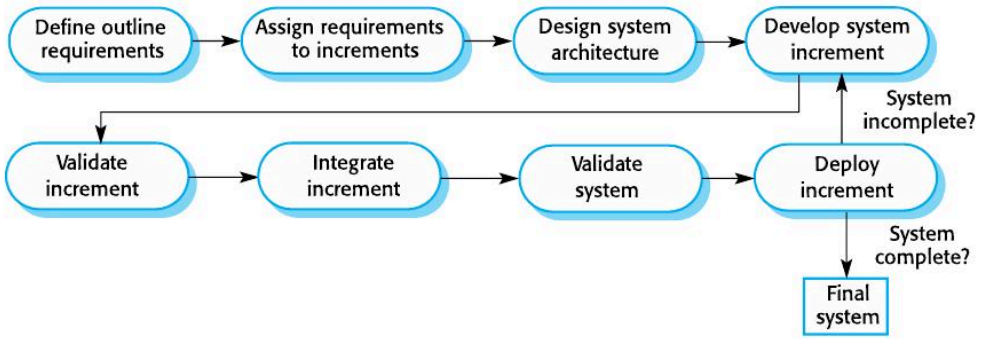
- **Incremental development** — 증분 단위로 개발하고 다음 증분 전에 평가
  - 애자일 방법의 일반적 방식, 평가는 사용자·고객 대리인이
- **Incremental delivery** — 증분을 최종 사용자가 쓰게 배포
  - 실제 사용에 대한 더 현실적인 평가
  - 기존 시스템을 **대체**하는 경우엔 어렵다 — 증분 기능이 대체될 시스템보다 적으니까

필기 ✍️

⚠ 주의 development(개발해서 평가) vs delivery(실제 배포) — 이름 비슷, 구분!

Coping with change

Incremental development and delivery



Chapter 2 Software Processes 54

점증적 인도 (그림)

- 요구 개요 정의 → 증분에 요구 배정 → 시스템 아키텍처 설계 → (증분 개발 → 검증 → 통합 → 시스템 검증 → 배포) 반복 → 최종 시스템

필기 ✍️

— 넘어가도 되는 쪽

Coping with change

Incremental development and delivery

[Incremental delivery advantages]

- Customer value can be delivered with each increment so system **functionality** is **available earlier**.
- Early increments act as a prototype to **help elicit requirements** for later increments.
- **Lower risk** of overall project failure.
- The **highest priority system** services tend to receive the most testing.

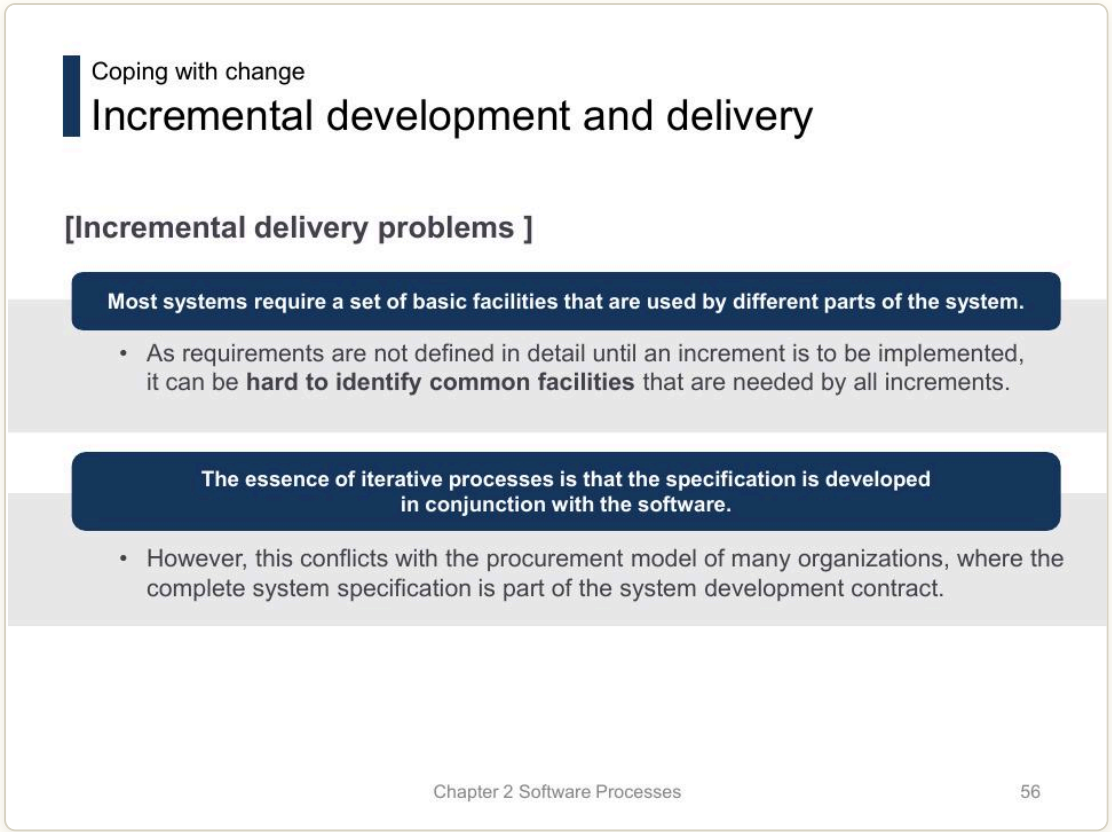
Chapter 2 Software Processes 55

점증적 인도의 장점

- 증분마다 고객 가치를 전달 → 기능을 **더 일찍** 사용
- 앞 증분이 **프로토타입** 역할 — 뒤 증분 요구 도출에 도움
- 전체 프로젝트 실패 **위험이 낮다**
- 우선순위 높은 서비스가 **가장 많이 테스트**된다

필기 ✍️

★ 시험 장점 4개

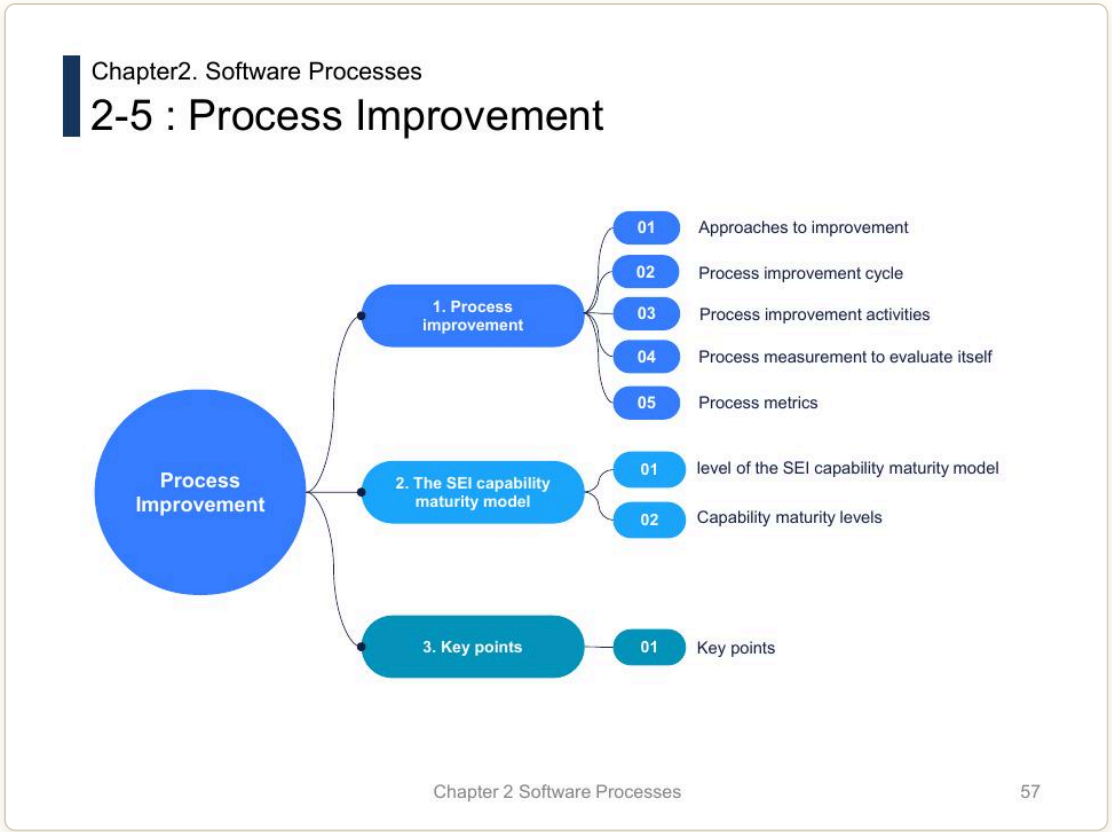


점증적 인도의 문제

- 대부분 시스템은 여러 부분이 쓰는 **공통 기본 기능**이 필요
  - 증분을 구현할 때까지 요구를 자세히 정하지 않으니, 모든 증분에 필요한 공통 기능을 찾기 어렵다
- 반복 프로세스의 본질은 명세를 SW와 **함께** 개발하는 것
  - 그런데 이는 **전체 명세가 개발 계약의 일부**인 많은 조직의 조달 모델과 충돌

필기 ✍️

★ 시험 문제 2개: 공통 기능 찾기 어려움 · 계약(조달) 모델과 충돌



2-5: 프로세스 개선

- 개선 접근법, 개선 주기, 개선 활동, 측정, 지표
- SEI 능력 성숙도 모델(CMM)
- 핵심 정리

필기 ✍️

💡 이해 절 표지 — 아래 쪽들의 지도

Process improvement

Process improvement : 프로세스 개선

- Many software companies have turned to **software process improvement** as a way of **enhancing** the **quality** of their software, **reducing costs** or **accelerating** their **development** processes.

- **Process improvement** means **understanding existing processes** and **changing these processes** to increase product quality and/or reduce costs and development time.

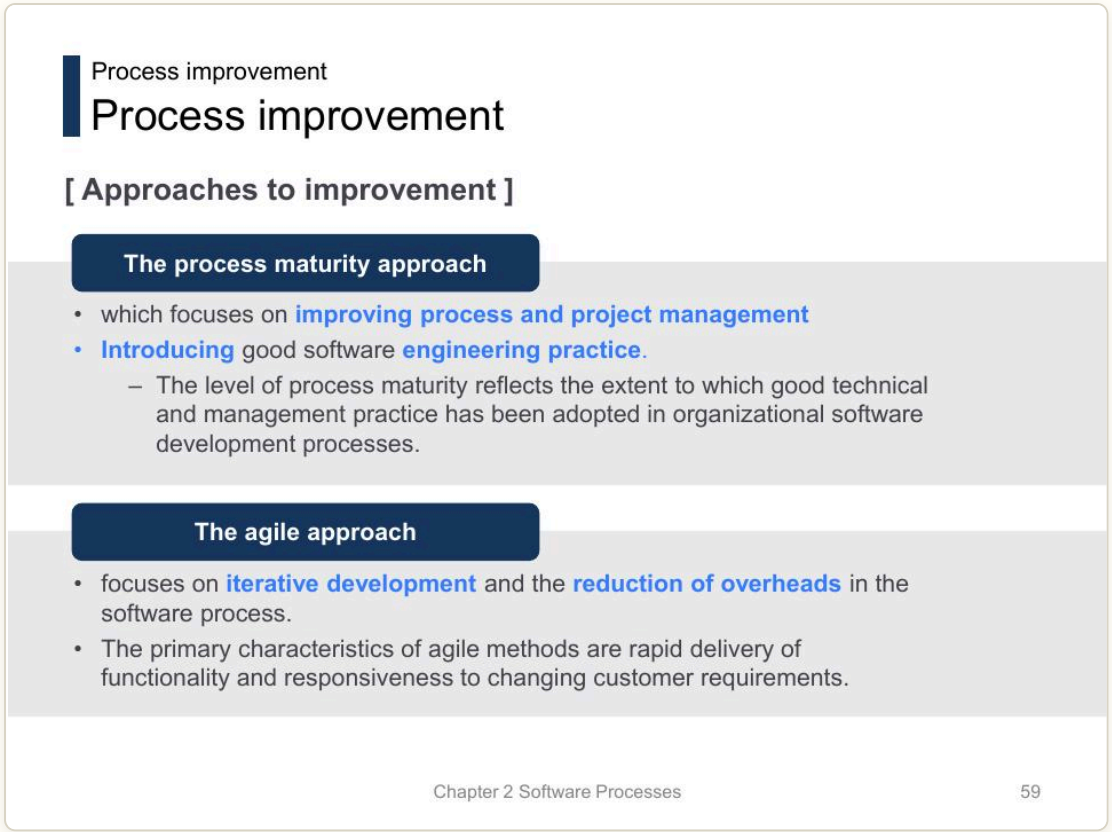
Chapter 2 Software Processes 58

프로세스 개선

- 많은 SW 회사가 품질을 높이고, 비용을 줄이고, 개발을 빠르게 하려고 **SW 프로세스 개선**에 나섬
- 프로세스 개선 = 기존 프로세스를 **이해**하고, 품질↑·비용↓·개발 시간↓를 위해 **바꾸는** 것

필기 ✍️

— 넘어가도 되는 쪽

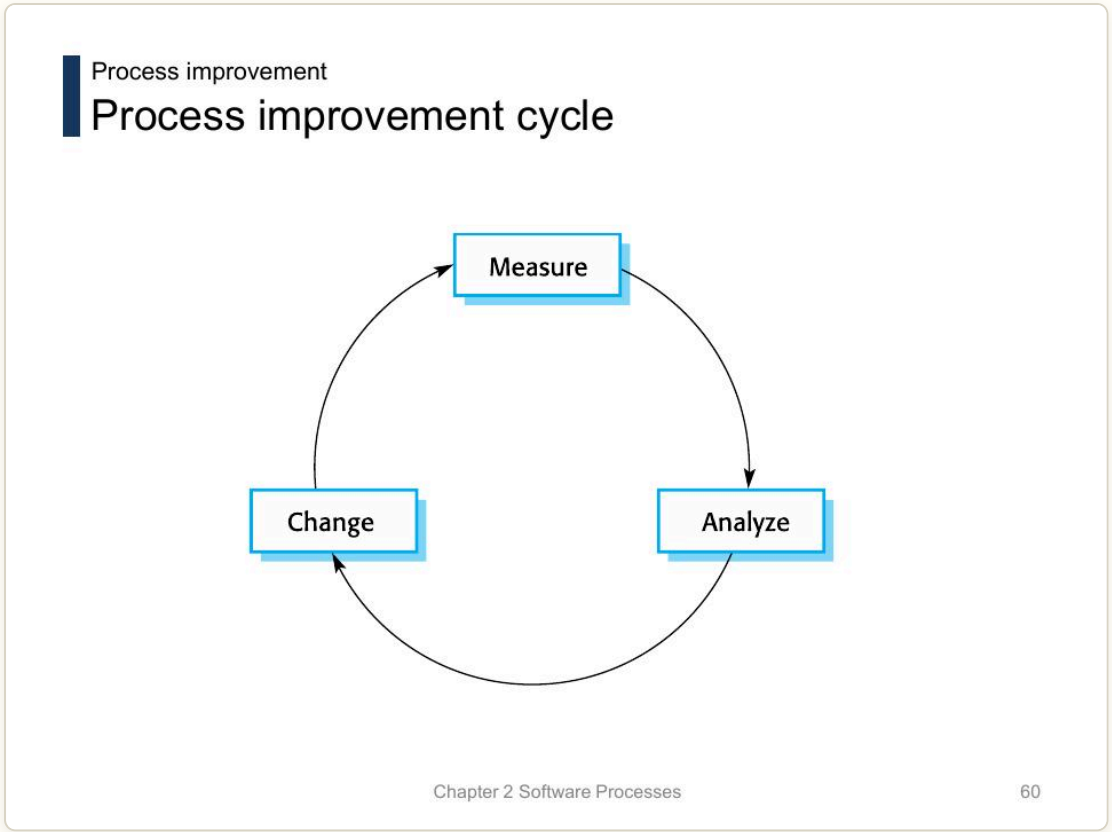


개선 접근법

- **The process maturity approach (성숙도 접근)**
  - 프로세스와 프로젝트 **관리** 개선, 좋은 SE 실천 도입
  - 성숙도 수준 = 좋은 기술·관리 실천을 조직 프로세스에 얼마나 받아들였나
- **The agile approach (애자일 접근)**
  - 반복 개발과 프로세스 **오버헤드 감소**에 집중
  - 핵심 특징: 기능의 빠른 전달, 바뀌는 고객 요구에 대한 대응

필기 ✍️

★ 시험 개선 접근 2개: 성숙도(관리·실천) vs 애자일(오버헤드↓)



프로세스 개선 주기 (그림)

- Measure(측정) → Analyze(분석) → Change(변경) → 다시 측정 ...

필기 ✍️

— 넘어가도 되는 쪽



프로세스 개선 활동

- **Process measurement (측정)** — 프로세스나 제품의 속성을 하나 이상 측정
  - 측정값이 **기준선(baseline)**이 되어 개선 효과를 판단
- **Process analysis (분석)** — 현재 프로세스를 평가해 약점과 병목을 찾음
  - 프로세스를 기술하는 프로세스 모델(process map)을 만들 수도
- **Process change (변경)** — 찾은 약점을 고치는 변경을 제안·도입
  - 그리고 다시 주기를 돌며 변경 효과 데이터를 모음

필기 ✍️

★ 시험 개선 주기: Measurement → Analysis → change

Process improvement

Process improvement

[Process measurement to evaluate itself]

- Wherever **possible, quantitative process data** should be collected
  - However, where organisations do not have clearly defined process standards this is very difficult as you don't know what to measure.
  - A process may have to be defined before any measurement is possible.
- Process measurements should be used to assess process improvements
  - But this does not mean that measurements should drive the improvements.
  - The improvement driver should be the organizational objectives.

Chapter 2 Software Processes 62

프로세스 측정

- 가능하면 **정량적** 프로세스 데이터를 모아야
  - 하지만 프로세스 표준이 명확하지 않은 조직은 무엇을 잴지 몰라 매우 어렵다
  - 측정 전에 프로세스를 **먼저 정의**해야 할 수도
- 측정은 개선을 **평가**하는 데 써야 한다
  - 하지만 측정이 개선을 **이끌어서는** 안 된다
  - 개선의 동인은 **조직의 목표**여야

필기 ✍️

⚠ 주의 '측정이 개선을 주도해야 한다' ⟶ X (주도는 조직 목표)

Process improvement

Process improvement

[Process metrics]

- Time taken for process activities to be completed
  - E.g. Calendar time or effort to complete an activity or process.
- Resources required for processes or activities
  - E.g. Total effort in person-days.
- Number of occurrences of a particular event
  - E.g. Number of defects discovered.

Chapter 2 Software Processes 63

프로세스 지표 (metrics)

- 프로세스 활동을 끝내는 데 걸린 **시간** — 예: 달력 시간, 노력
- 프로세스·활동에 필요한 **자원** — 예: 총 노력(person-days)
- 특정 **사건의 발생 수** — 예: 발견된 결함 수

필기 ✍️

★ 시험 지표 3종: 시간 · 자원 · 사건 수

Process improvement

The SEI capability maturity model

[Level of the SEI capability maturity model]

Initial	Essentially uncontrolled
Repeatable	Product management procedures defined and used
Defined	Process management procedures and strategies defined and used
Managed	Quality management strategies defined and used
Optimising	Process improvement strategies defined and used

Chapter 2 Software Processes 64

SEI 능력 성숙도 모델 — 수준

- **Initial** — 사실상 통제되지 않음
- **Repeatable** — 제품 관리 절차가 정의되고 쓰임
- **Defined** — 프로세스 관리 절차와 전략이 정의되고 쓰임
- **Managed** — 품질 관리 전략이 정의되고 쓰임
- **Optimising** — 프로세스 개선 전략이 정의되고 쓰임

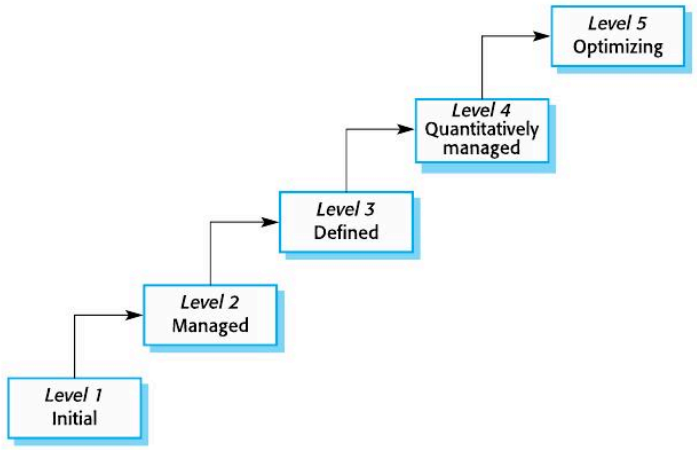
필기 ✍️

★ 시험 CMM 5단계(이 쪽): Initial ⟶ Repeatable ⟶ Defined ⟶ Managed ⟶ Optimising

Process improvement

The SEI capability maturity model

[Capability maturity levels]



Chapter 2 Software Processes 65

능력 성숙도 수준 (그림)

- Level 1 **Initial** → Level 2 **Managed** → Level 3 **Defined** → Level 4 **Quantitatively managed** → Level 5 **Optimizing**

필기 ✍️

⚠ 주의 p.64(옛 CMM: 2=Repeatable, 4=Managed)와 p.65(CMMI: 2=Managed, 4=Quantitatively managed) 이름이 다르다! 문제에서 어느 쪽을 묻는지 확인

Process improvement

Key points

- Software processes are the activities involved in producing a software system. Software process models are abstract representations of these processes.
- General process models describe the organization of software processes.
  - Examples of these general models include the 'waterfall' model, incremental development, and reuse-oriented development.
- Requirements engineering is the process of developing a software specification.

Chapter 2 Software Processes 66

핵심 정리 ①

- SW 프로세스는 SW 시스템 생산에 관련된 활동들, 프로세스 모델은 그 추상적 표현
- 일반 프로세스 모델은 SW 프로세스의 조직을 기술 — 폭포수, 점증적 개발, 재사용 지향 개발
- 요구공학은 SW 명세를 개발하는 과정

필기 ✍️

— 넘어가도 되는 쪽

Process improvement

Key points

- ✧ Design and implementation processes are concerned with transforming a requirements specification into an executable software system.
- ✧ Software validation is the process of checking that the system conforms to its specification and that it meets the real needs of the users of the system.
- ✧ Software evolution takes place when you change existing software systems to meet new requirements. The software must evolve to remain useful.
- ✧ Processes should include activities such as prototyping and incremental delivery to cope with change.

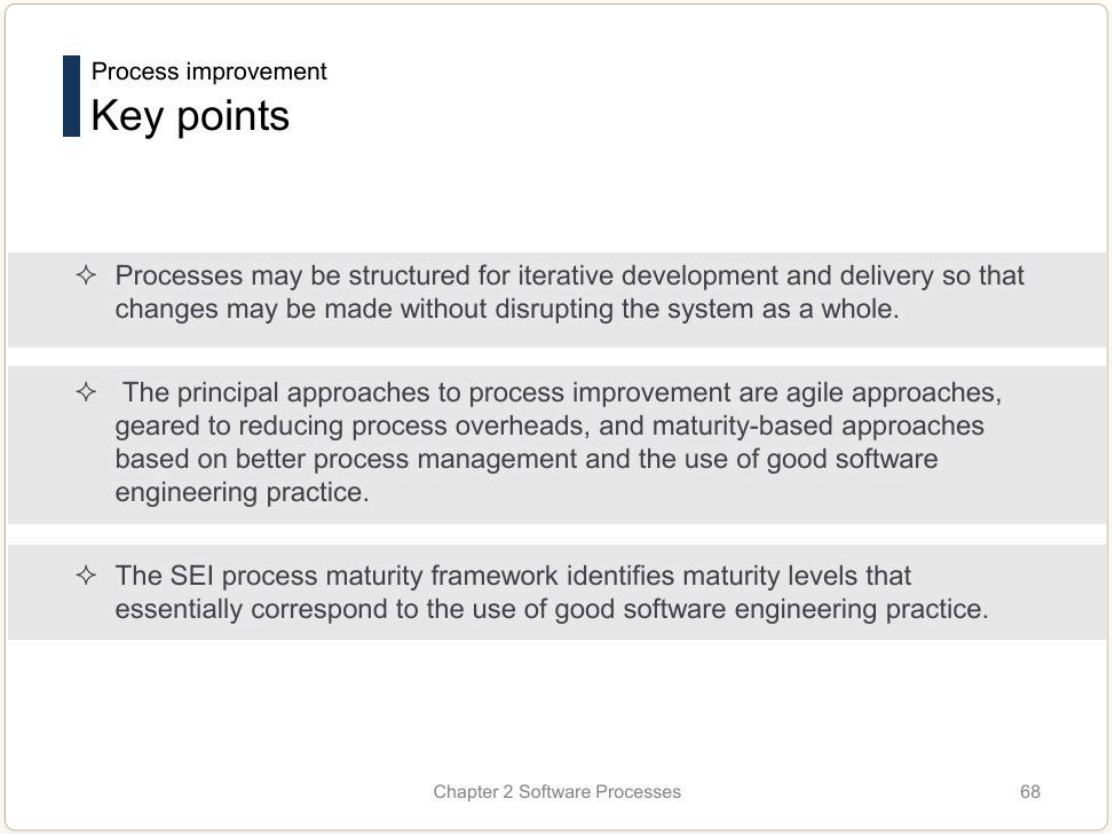
Chapter 2 Software Processes 67

핵심 정리 ②

- 설계·구현은 요구 명세를 실행 가능한 시스템으로 바꾸는 과정
- SW 검증은 시스템이 명세에 맞고 사용자의 진짜 요구를 만족하는지 확인하는 과정
- SW 진화는 기존 시스템을 새 요구에 맞게 바꿀 때 — SW는 쓸모를 유지하려면 진화해야
- 프로세스에는 변경 대처를 위한 프로토타이핑, 점증적 인도 같은 활동이 있어야

필기 ✍️

— 넘어가도 되는 쪽



핵심 정리 ③

- 반복 개발·인도로 프로세스를 구성하면 시스템 전체를 흔들지 않고 변경 가능
- 프로세스 개선의 주요 접근: 오버헤드를 줄이는 **애자일** / 더 나은 관리와 좋은 SE 실천에 기반한 **성숙도** 접근
- SEI 성숙도 프레임워크는 좋은 SE 실천 사용에 해당하는 성숙도 수준을 정의

필기 ✍️

— 넘어가도 되는 쪽